

# 教学安排（星期四，08:00-09:35，六教6A116）



| 周次 | 日期    | 内容          | 作业 | 编程 | 项目 | 内容                    |
|----|-------|-------------|----|----|----|-----------------------|
| 1  | 2月29日 | 绪论          |    |    |    | 绪论，课程内容               |
| 2  | 3月 7日 | 无信息搜索       |    | 1  |    | 状态空间表示，宽度优先，深度优先      |
| 3  | 3月14日 | 有信息搜索（1）    |    |    | 1  | 算法复杂度分析，一致代价，贪婪最佳优先搜索 |
| 4  | 3月21日 | 有信息搜索（2）    | 1  |    |    | A* 算法                 |
| 5  | 3月28日 | 约束满足        | 2  |    |    | 约束满足问题，回溯法，局部搜索       |
| 6  | 4月 4日 | 清明节假期       |    |    |    |                       |
| 7  | 4月11日 | 对抗搜索        | 3  |    |    | 博弈树搜索，蒙特卡洛树搜索         |
| 8  | 4月18日 | 逻辑推理        | 4  |    |    | 归结原理                  |
| 9  | 4月25日 | 线性回归        |    |    | 2  | 线性回归                  |
| 11 | 5月 9日 | Logistic 回归 | 5  |    |    | Logistic 回归           |
| 11 | 5月11日 | 前馈神经网络      |    | 2  |    | 前馈神经网络，机器学习项目         |
| 12 | 5月16日 | 卷积神经网络      | 6  |    |    | 卷积神经网络                |
| 13 | 5月23日 | 马尔可夫决策过程    | 7  |    |    | 强化学习的数学基础             |
| 14 | 5月30日 | 策略迭代与价值迭代   |    | 3  |    | 状态转移概率已知时的预测与控制       |
| 15 | 6月 6日 | 蒙特卡洛与时序差分   | 8  |    |    | 状态转移概率未知时的预测与控制       |
| 16 | 6月13日 | 深度强化学习      |    |    |    | 价值函数的近似，深度强化学习，考试解读   |

问题求解  
逻辑推理  
机器学习

# 人工智能原理

Principles of Artificial Intelligence

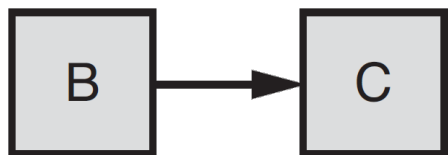


乔晖

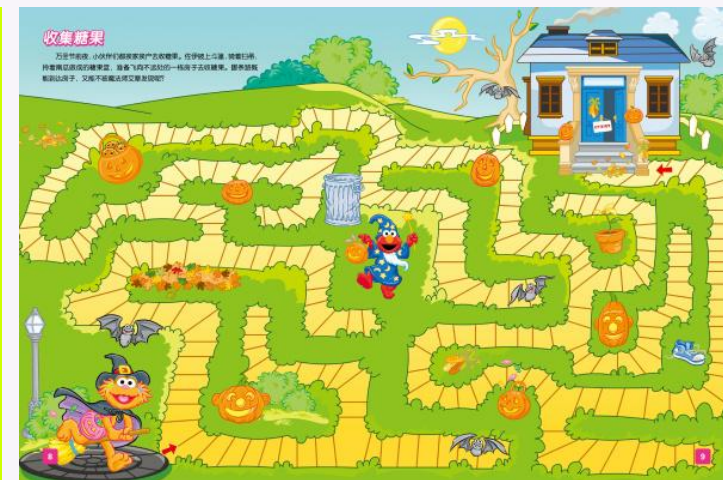
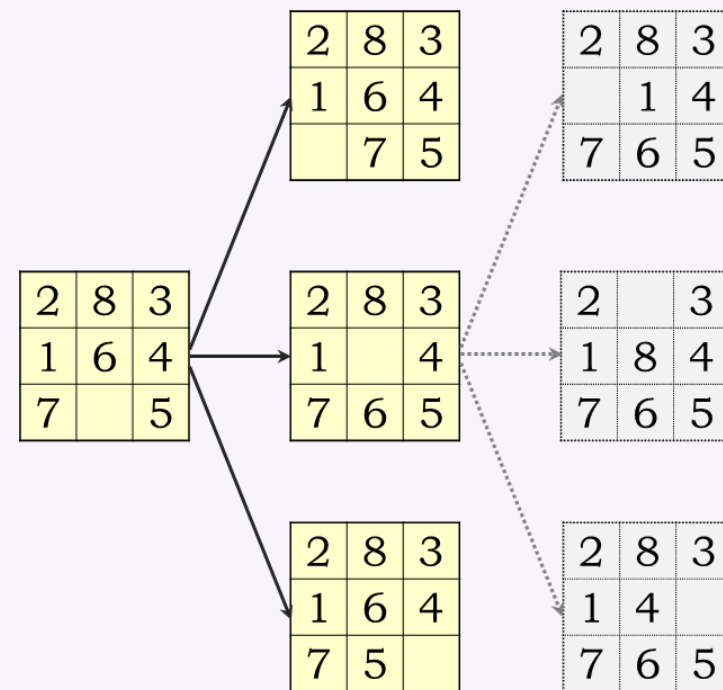
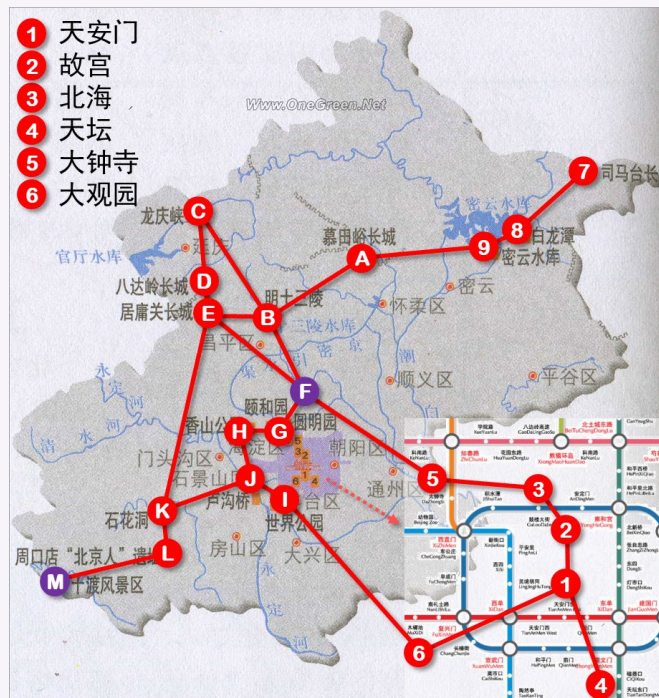
自动化系

# 原子表示

- ▶ 状态空间表示把状态当作黑盒子
- ▶ 优点：简单、通用、无领域知识
- ▶ 缺点：忽略了状态内部的信息
- ▶ 实例
  - ▶ 通用搜索算法
  - ▶ 马尔可夫决策过程



# Atomic representation



# 8 皇后问题

# Eight Queens puzzle

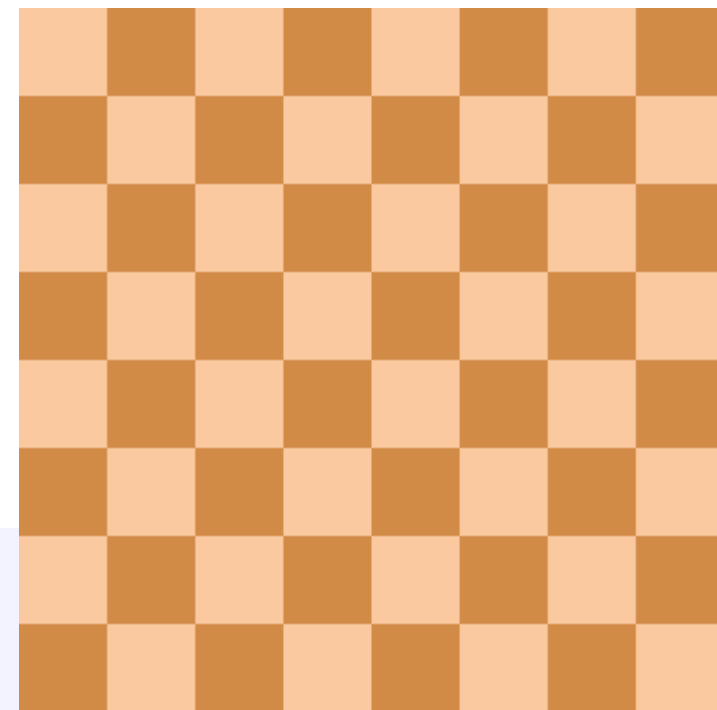
- ▶ 在  $8 \times 8$  的象棋棋盘上摆放 8 个皇后，使其不能互相攻击

- ▶ 原子表示法

- ▶ 状态：将 8 个皇后的一种摆法当作一种状态
  - ▶ 操作：将其中一个皇后移动到另外一个位置
  - ▶ 代价：不考虑

$$64^8 = 2^{48} = 281,474,976,710,656 \approx 2.8 \times 10^{14}$$

有没有更好的办法？



- ▶ 两个皇后不能放到同一位置

- ▶ 位置  $(x_i, y_i)$ ,  $x_i \neq x_j$  AND  $y_i \neq y_j$

$$64^8 \rightarrow 64 \times 63 \times \dots \times 57$$

- ▶ 两个皇后不能放到同一列

- ▶ 位置  $(x_i, i)$ ,  $x_i \neq x_j$

$$64 \times 63 \times \dots \times 57 \rightarrow 8!$$

用一组变量来描述状态，变量还有一定的约束



# 要素化表示

# Factored representation

- ▶ 用一组变量来描述一个状态
- ▶ 每个变量有自己的取值范围

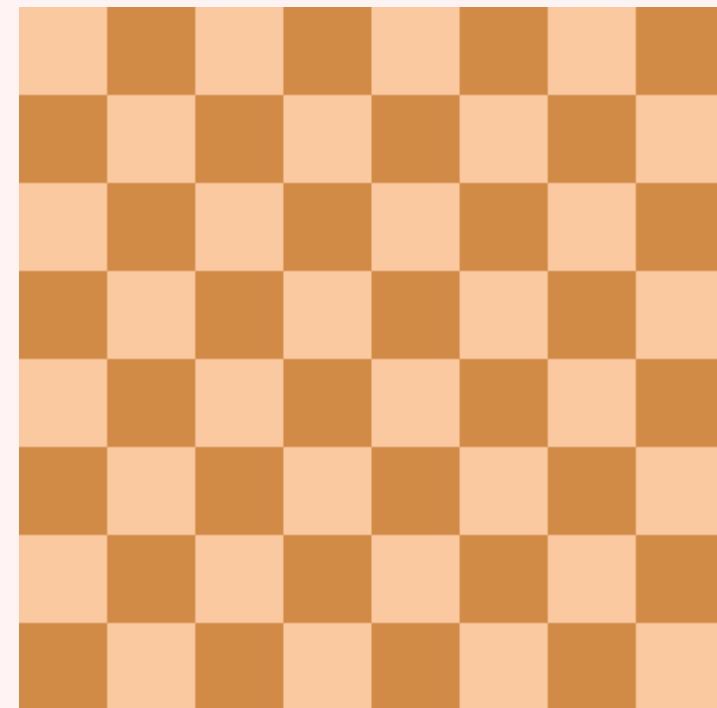
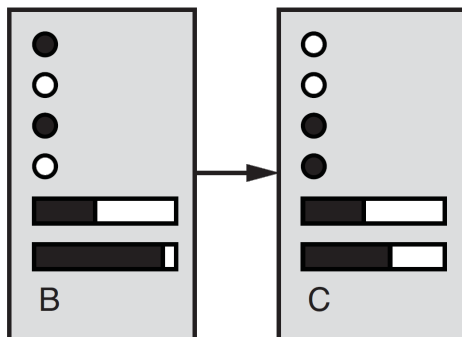
$$\mathbf{X} = \{X_1, \dots, X_n\}$$
$$\mathbf{D} = \{D_1, \dots, D_n\}, X_i \in D_i$$

要素化表示

## 8 皇后问题

- ▶ 在  $8 \times 8$  的象棋棋盘上摆放 8 个皇后，使其不能互相攻击
- ▶ 变量:  $\mathbf{X} = \{X_1, \dots, X_n\}, n = 8$
- ▶ 值域:  $\mathbf{D} = \{D_1, \dots, D_n\}, X_i \in D_i = \{1, \dots, n\}$

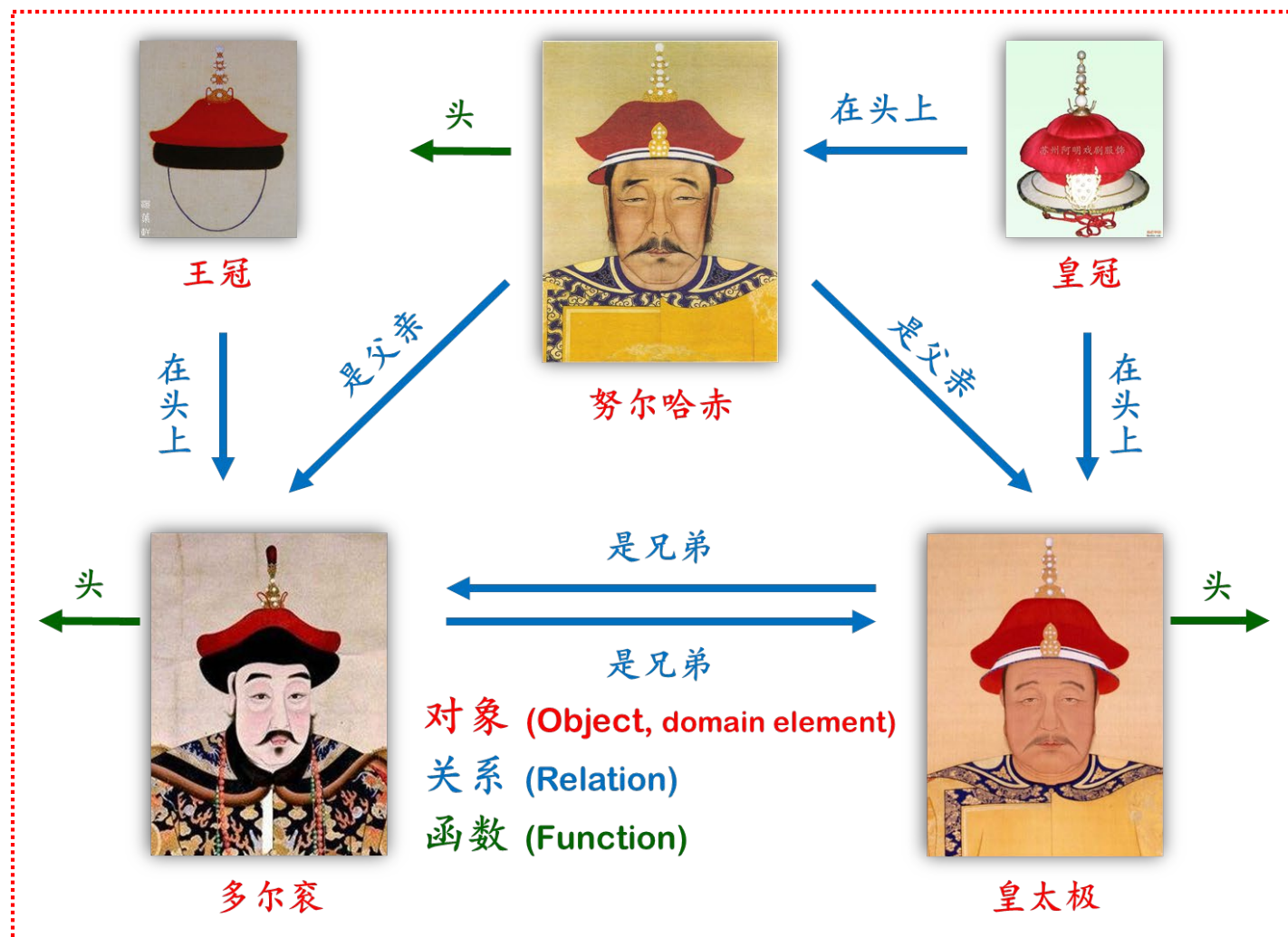
- ▶ 命题逻辑
- ▶ 经典规划
- ▶ 机器学习



# 结构化表示

# Structured representation

- ▶ 不仅需要变量来描述事物，还需要表示事物之间的关系
  - ▶ 一阶谓词逻辑
  - ▶ 关系型数据库



# 约束满足问题

Constraint Satisfaction Problem

|  |  |
|--|--|
|  |  |
|  |  |

# 约束满足问题

# Constraint satisfaction problem (CSP)

- ▶ 用一组变量来描述一个状态
- ▶ 每个变量有自己的取值范围
- ▶ 变量的取值还有一定的约束

$$\mathbf{X} = \{X_1, \dots, X_n\}$$

$$\mathbf{D} = \{D_1, \dots, D_n\}, X_i \in D_i$$

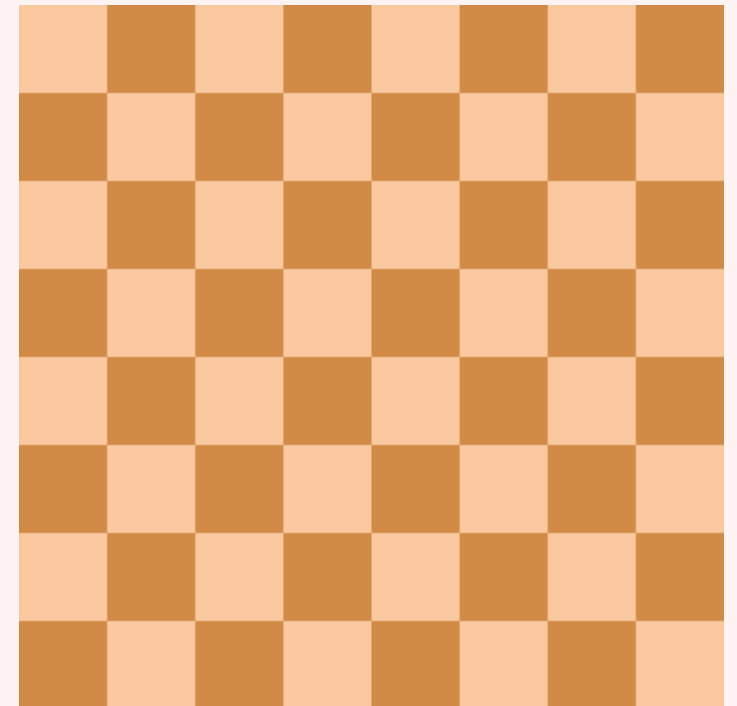
$$\mathbf{C} = \{(\text{scope}, \text{relation})\}$$

要素化表示

约束满足

## 8 皇后问题

- ▶ 在  $8 \times 8$  的象棋棋盘上摆放 8 个皇后，使其不能互相攻击
- ▶ 变量:  $\mathbf{X} = \{X_1, \dots, X_n\}, n = 8$
- ▶ 值域:  $\mathbf{D} = \{D_1, \dots, D_n\}, X_i \in D_i = \{1, \dots, n\}$
- ▶ 约束:  $\mathbf{C} = \{((x_i, x_j), x_i \neq x_j), ((x_i, x_j), |x_i - x_j| \neq |i - j|)\}$





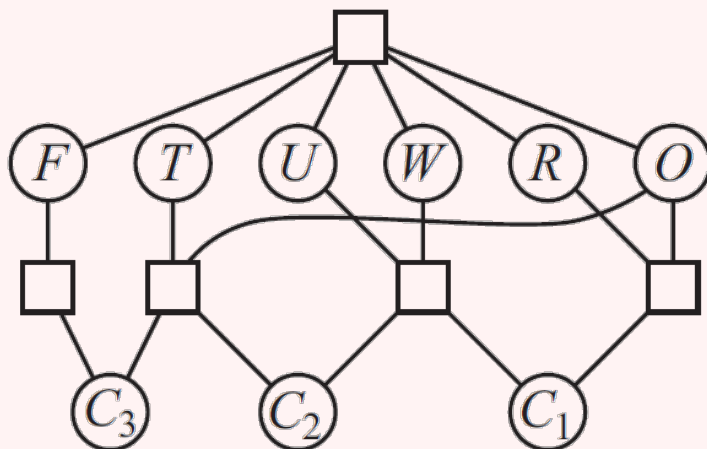
# 约束满足问题举例

## 算数密码

- ▶ 将字母换成数字，使得运算成立
- ▶ 变量:  $\mathbf{X} = \{X_1, \dots, X_n\}, n = 6$
- ▶ 值域:  $\mathbf{D} = \{D_1, \dots, D_n\}, X_i \in D_i = \{0, \dots, 9\}$
- ▶ 约束:  $O + O = R + 10 \cdot C_{10}$   
 $C_{10} + W + W = U + 10 \cdot C_{100}$   
 $C_{100} + T + T = O + 10 \cdot C_{1000}$   
 $C_{1000} = F$ ,

$$\begin{array}{r} C_{10} + W + W = U + 10 \cdot C_{100} \\ C_{100} + T + T = O + 10 \cdot C_{1000} \\ C_{1000} = F, \end{array}$$

$$\begin{array}{r} T \ W \ O \\ + \ T \ W \ O \\ \hline F \ O \ U \ R \end{array}$$



## 数独游戏

- ▶ 将数字填入空白格，要求每行、每列、每个方框内，每个数字只出现一次
- ▶ 变量:  $\mathbf{X} = \{X_1, \dots, X_n\}, n = 81$
- ▶ 值域:  $\mathbf{D} = \{D_1, \dots, D_n\}, X_i \in D_i = \{1, \dots, 9\}$
- ▶ 约束: 行、列、方框内的数字不能重复

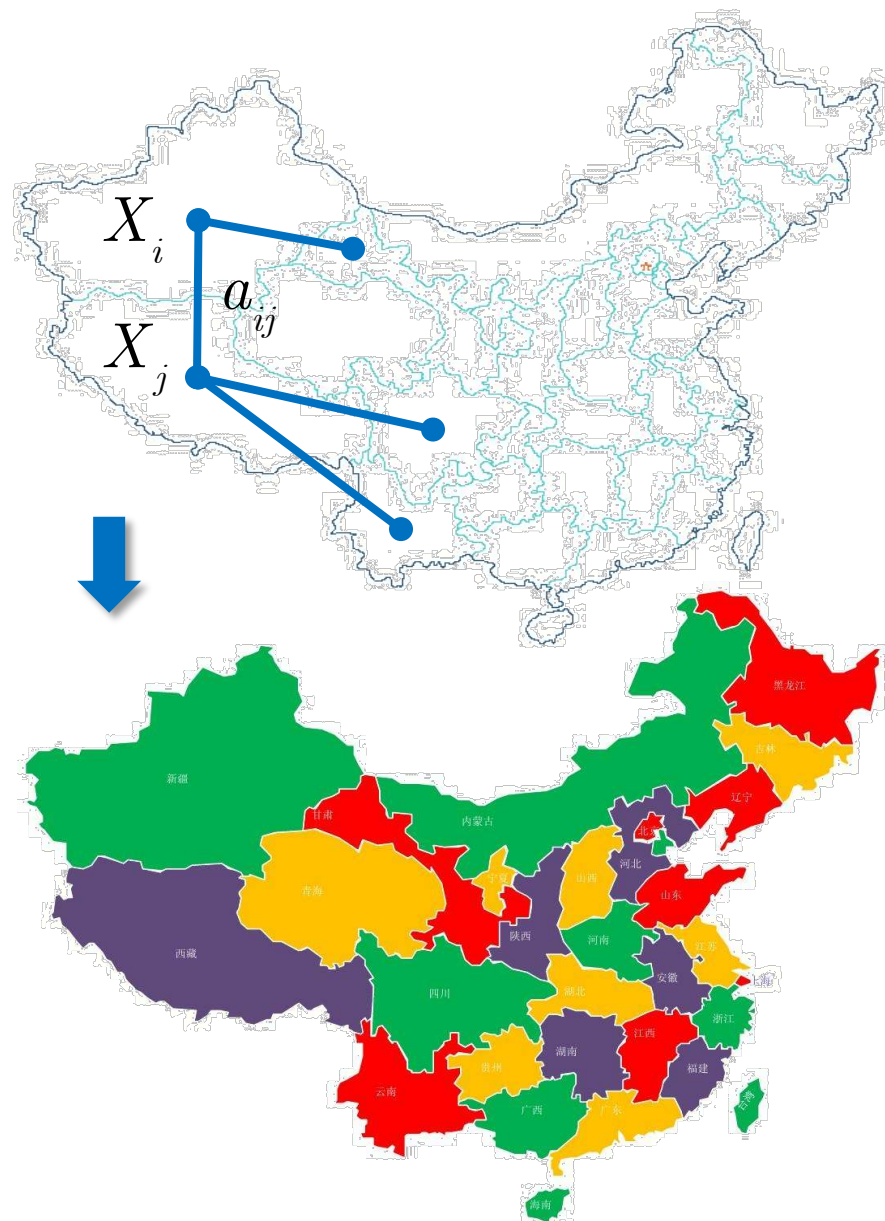
|   | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| A |   |   | 3 |   | 2 |   | 6 |   |   |
| B | 9 |   |   | 3 |   | 5 |   |   | 1 |
| C |   |   | 1 | 8 |   | 6 | 4 |   |   |
| D |   |   | 8 | 1 |   | 2 | 9 |   |   |
| E | 7 |   |   |   |   |   |   |   | 8 |
| F |   |   | 6 | 7 |   | 8 | 2 |   |   |
| G |   |   | 2 | 6 |   | 9 | 5 |   |   |
| H | 8 |   |   | 2 |   | 3 |   |   | 9 |
| I |   |   | 5 |   | 1 |   | 3 |   |   |



|   | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| A | 4 | 8 | 3 | 9 | 2 | 1 | 6 | 5 | 7 |
| B | 9 | 6 | 7 | 3 | 4 | 5 | 8 | 2 | 1 |
| C | 2 | 5 | 1 | 8 | 7 | 6 | 4 | 9 | 3 |
| D | 5 | 4 | 8 | 1 | 3 | 2 | 9 | 7 | 6 |
| E | 7 | 2 | 9 | 5 | 6 | 4 | 1 | 3 | 8 |
| F | 1 | 3 | 6 | 7 | 9 | 8 | 2 | 4 | 5 |
| G | 3 | 7 | 2 | 6 | 8 | 9 | 5 | 1 | 4 |
| H | 8 | 1 | 4 | 2 | 5 | 3 | 7 | 6 | 9 |
| I | 6 | 9 | 5 | 4 | 1 | 7 | 3 | 8 | 2 |

## 地图着色

- ▶ 用四种颜色给地图着色，相邻区域的颜色不相同
- ▶ 变量： $\mathbf{X} = \{X_1, \dots, X_n\}$
- ▶ 值域： $\mathbf{D} = \{D_1, \dots, D_n\}, X_i \in D_i = \{1, 2, 3, 4\}$
- ▶ 约束： $x_i \neq x_j, \text{ if } a_{ij} = 1$



## 作业调度

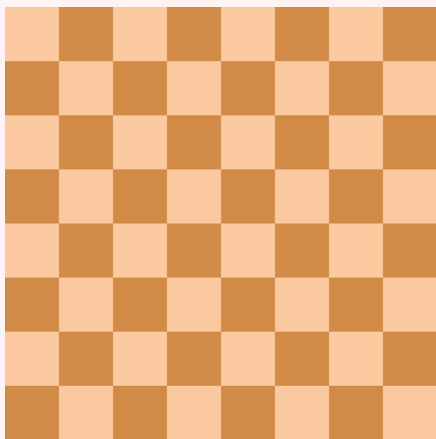
- ▶ 汽车组装过程中各个任务如何安排
- ▶ 变量:  $\mathbf{X} = \{X_1, \dots, X_n\}$
- ▶ 值域:  $\mathbf{D} = \{D_1, \dots, D_n\}, X_i \in D_i = \{1, \dots, 30\}$
- ▶ 约束:  $T_i + d_i \leq T_j$ 
  - ▶ 任务  $i$  要在任务  $j$  之前完成
  - ▶ 执行任务  $i$  需要时间  $d_i$
- ▶ 这一类问题约束可能会很多, 很琐碎
- ▶ 即便约束变量有几千个, 也能使用约束满足问题算法求解



汽车组装有许多任务组成, 每个任务当作一个变量, 变量的值是任务开始的时间, 用整数分钟表示。约束是某个任务必须在另一个任务之前这一类的限制

## 离散型

- 变量取**有限**个离散值



- 变量取**无限**个离散值

- 整数规划
- 背包问题

$$\begin{aligned} \text{Max } Z &= 5x_1 + 8x_2 \\ \text{s.t. } x_1 + x_2 &\leq 6 \\ 5x_1 + 9x_2 &\leq 45 \\ x_1, x_2 &\geq 0 \text{ integer} \end{aligned}$$

## 连续型

- 变量取连续值
- 线性规划
- 二次规划

$$\begin{aligned} \max z &= 2x_1 + 3x_2 - 5x_3 \\ \text{s.t. } x_1 + x_2 + x_3 &= 7 \\ 2x_1 - 5x_2 + x_3 &\geq 10 \\ x_1 + 3x_2 + x_3 &\leq 12 \\ x_1, x_2, x_3 &\geq 0 \end{aligned}$$

$$\begin{aligned} \min \quad q(x) &= \frac{1}{2} x^T G x + g^T x, \\ \text{s.t. } a_i^T x &= b_i, \quad i \in E, \\ a_i^T x &\geq b_i, \quad i \in I. \end{aligned}$$

# 约束满足问题分类

## 按约束分

### 一元约束

- ▶ 仅涉及一个变量的约束

- ▶ 北京必须是红色

$$C = ((c_{Beijing}), c_{Beijing} = \text{Red})$$

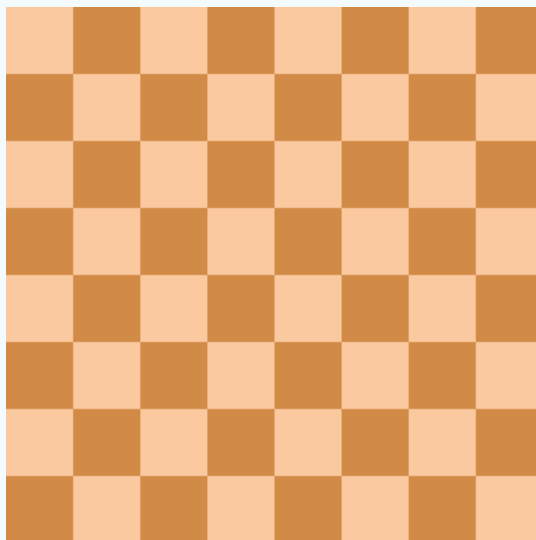


### 二元约束

- ▶ 涉及两个变量的约束

- ▶ 两个皇后不能在同行

$$C = ((x_i, x_j), x_i \neq x_j)$$



### 全局约束

- ▶ 涉及多个变量的约束

- ▶ 两个数字不能相同

$$C = ((d_{A1}, d_{A2}, \dots, d_{A9}), \text{Alldiff})$$

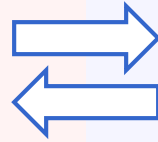
|   | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| A |   |   | 3 |   | 2 |   | 6 |   |   |
| B | 9 |   |   | 3 |   | 5 |   |   | 1 |
| C |   |   | 1 | 8 |   | 6 | 4 |   |   |
| D |   |   | 8 | 1 |   | 2 | 9 |   |   |
| E | 7 |   |   |   |   |   |   |   | 8 |
| F |   |   | 6 | 7 |   | 8 | 2 |   |   |
| G |   |   | 2 | 6 |   | 9 | 5 |   |   |
| H | 8 |   |   | 2 |   | 3 |   |   | 9 |
| I |   |   | 5 |   | 1 |   | 3 |   |   |

## 搜索

- ▶ 通过一定的搜索算法来对变量赋值

赋值

- ▶ 回溯搜索
- ▶ 智能回溯
- ▶ 局部搜索



## 推理

- ▶ 使用约束来减少变量的取值范围可能影响约束中其他变量的取值

约束传播

- ▶ 节点相容
- ▶ 边相容
- ▶ 路径相容
- ▶  $k$ -相容
- ▶ 全局约束



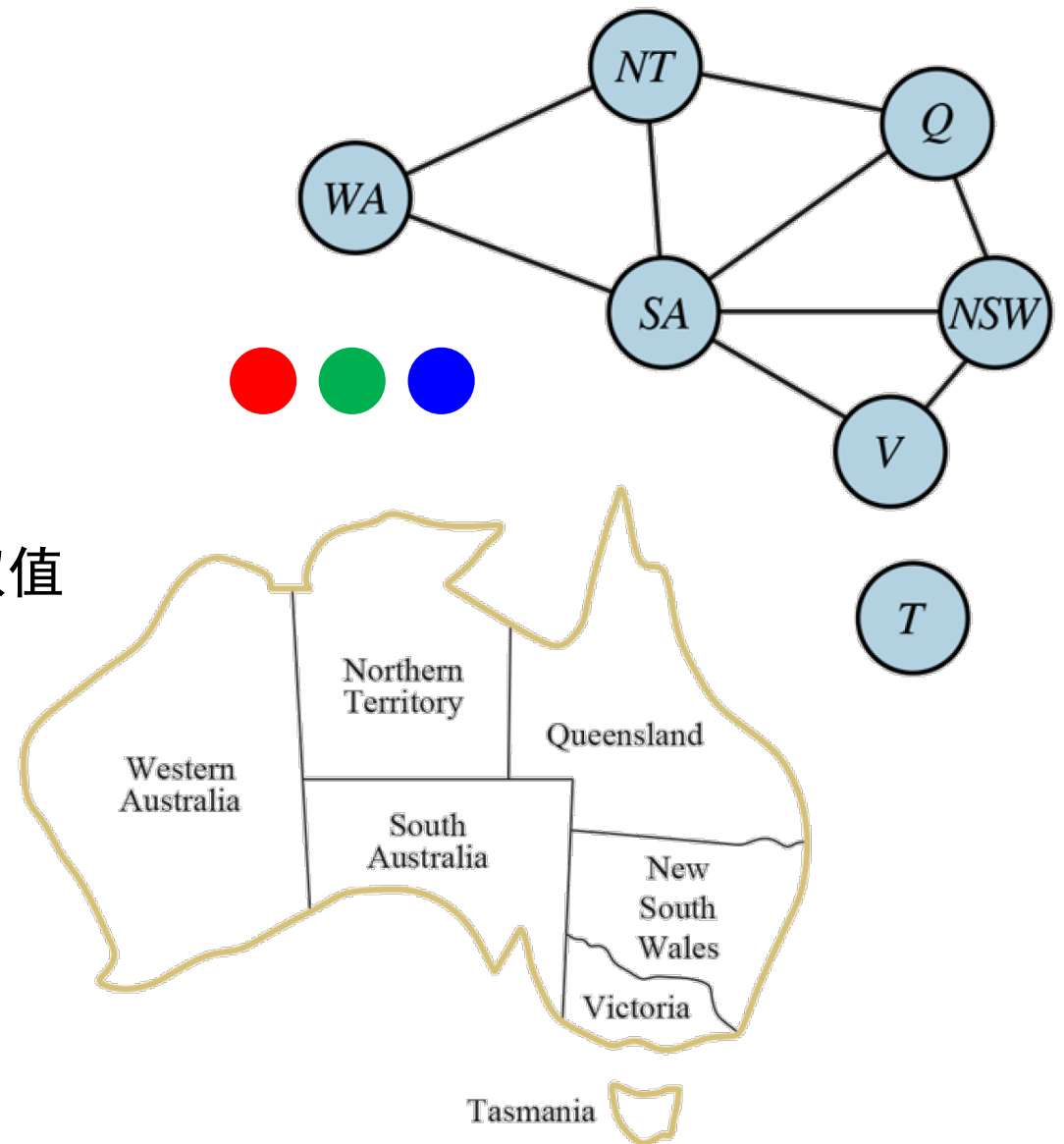
# 约束满足问题中的搜索

Searching for CSP problems

|  |  |
|--|--|
|  |  |
|  |  |

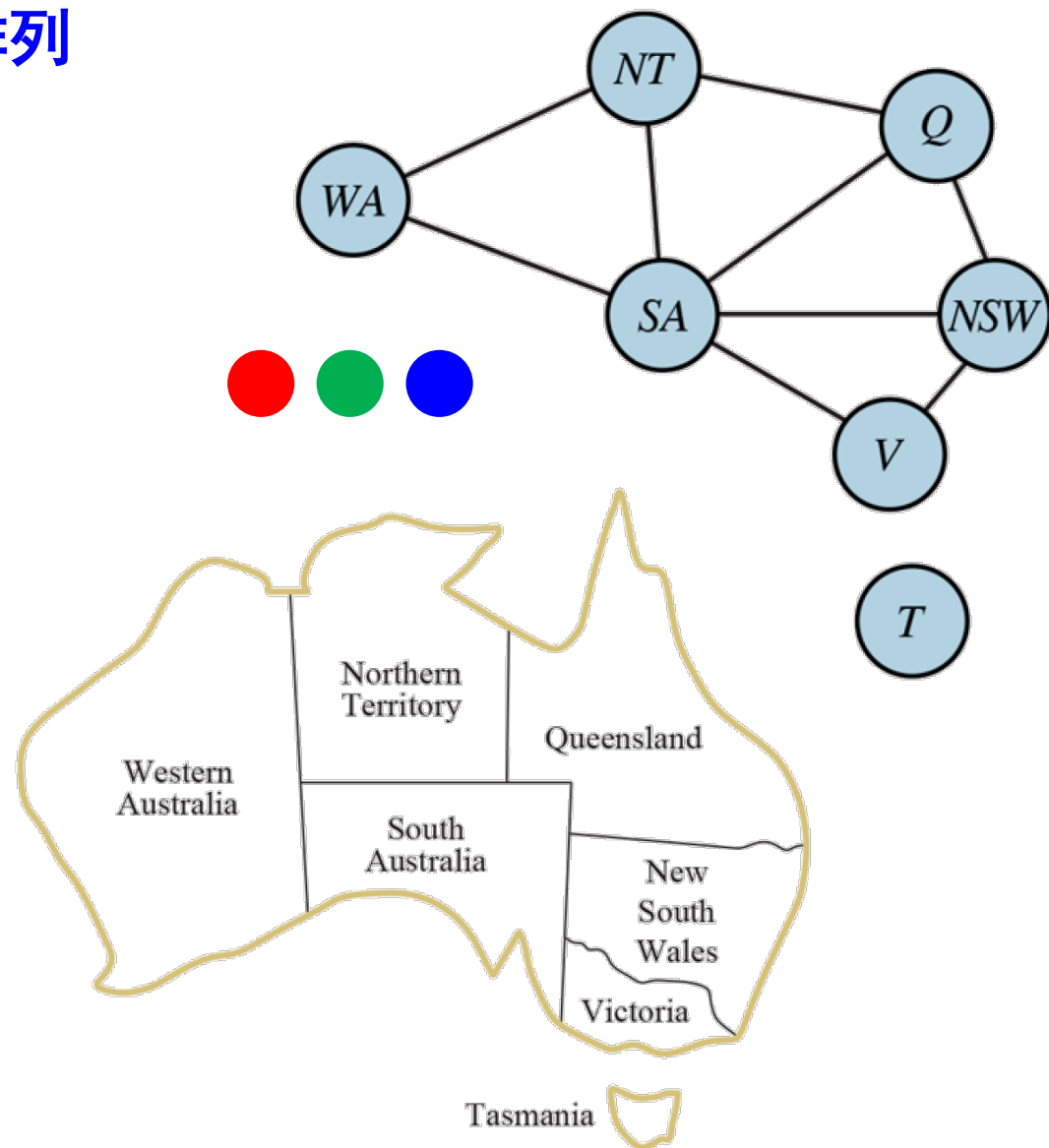
# 状态空间

- ▶ 状态：变量的部分赋值
  - ▶ 初始状态：没有变量赋值
  - ▶ 目标状态：所有变量赋值
- ▶ 行动：对某个变量赋值
- ▶ 代价：不考虑
- ▶ 问题：会发生指数爆炸
  - ▶ 第一层：分支  $nd$ ,  $n$  个变量，每个变量  $d$  种取值
  - ▶ 第二层：分支  $(n-1)d$
  - ▶ ...
  - ▶ 第  $n$  层：节点数  $n!d^n$
- ▶ 即便是8皇后问题，节点数也有  $8! \times 8^8 \sim 10^{12}$



# 状态空间的简化

- ▶ **变量赋值的顺序不重要，不需要考虑变量的排列**
- ▶ 状态：变量的部分赋值
- ▶ **行动：对固定顺序的变量赋值**
- ▶ 代价：不考虑
- ▶ 赋值过程
  - ▶ 第一层：分支  $d$ ，第一个变量，有  $d$  种取值
  - ▶ 第二层：分支  $d^2$ ，第二个变量，有  $d$  种取值
  - ▶ ...
  - ▶ 第  $n$  层：节点数  $d^n$
- ▶ 同样是8皇后问题，节点数减少到  $8^8 \sim 10^7$



## 无信息搜索

- ▶ 亦称盲目搜索
- ▶ 仅使用状态空间提供的信息
- ▶ 宽度优先搜索 Breadth-first search
- ▶ 一致代价搜索 Uniform-cost search
- ▶ 深度优先搜索 Depth-first search
- ▶ 深度受限搜索 Depth-limited search
- ▶ 迭代加深搜索 Iterative deepening search

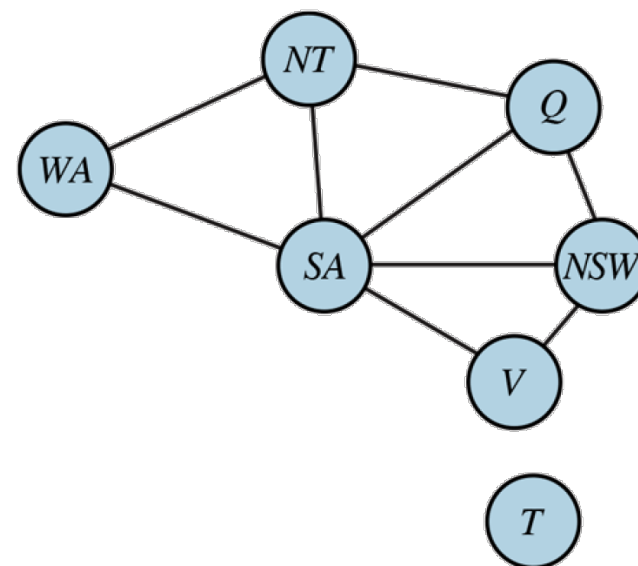
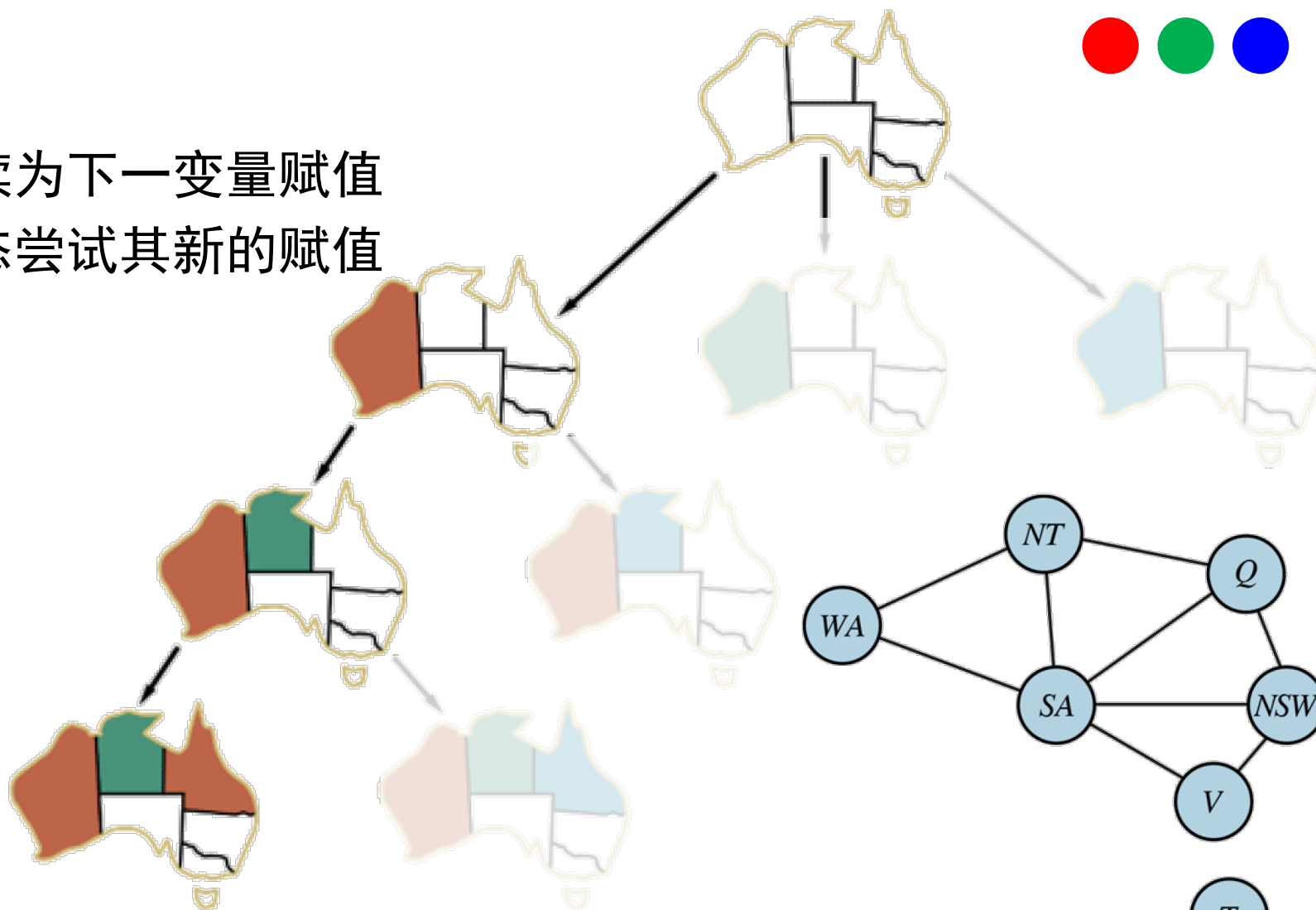
## 有信息搜索

- ▶ 亦称启发式搜索
- ▶ 还使用状态空间之外的信息
- ▶ 贪婪最佳优先搜索 Greedy best-first search
- ▶ A\* 算法 A\* Algorithm

# 回溯搜索

# Backtracking search

- ▶ 深度优先搜索的特殊版本
- ▶ 每次选择一个变量来赋值
  - ▶ 取合法值时从该状态继续为下一变量赋值
  - ▶ 没有合法值时返回父状态尝试其新的赋值



# 回溯搜索算法

# Backtracking search

**ALGORITHM** BACKTRACK(*assignment*, *csp*)  
**INPUT** An assignment of the parameters and a CSP problem  
**OUTPUT** A solution or failure

**IF** *assignment* is complete **THEN RETURN** *assignment*  
Select a variable {*var*} to assign value  
**FOR** each value of the selected variable **DO**  
    **IF** the assigned value is valid **THEN**  
        add {*var* = *value*} to *assignment*  
        *result*  $\leftarrow$  BACKTRACK(*assignment*, *csp*)  
        **IF** *result*  $\neq$  failure **THEN RETURN** *result*  
        Remove {*var* = *value*} from *assignment*  
**RETURN** failure

按照什么顺序来  
选择待赋值变量

按照什么顺序来  
选择该变量的值

回溯是不是可以  
做得更聪明一些

**ALGORITHM** BACKTRACKING-SEARCH(*csp*)  
**OUTPUT** A solution or failure

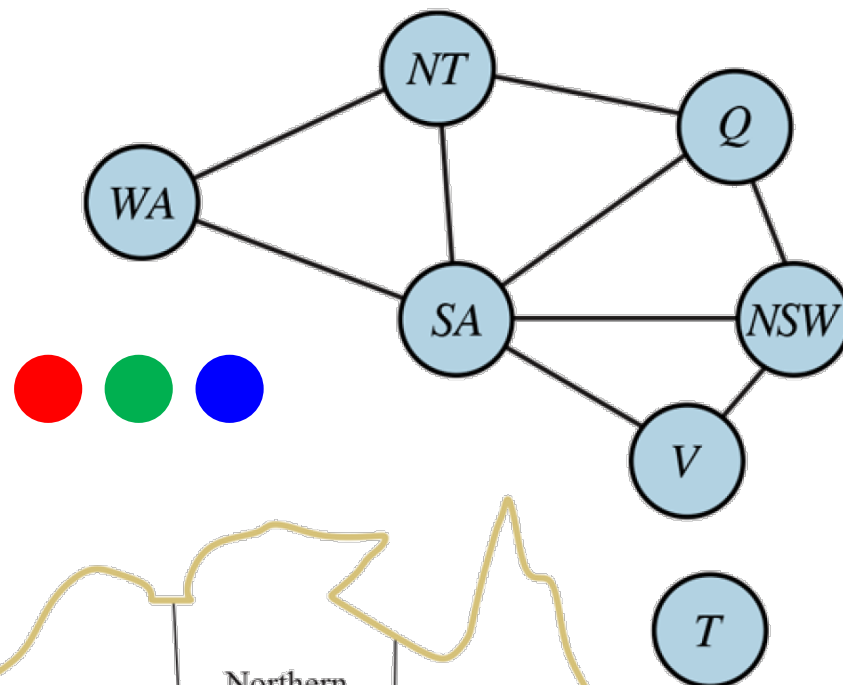
**RETURN** BACKTRACK({}, *csp*)



# 变量选择的顺序

## ▶ 静态选择：按照变量的固定顺序选择

- ▶ 简单、可能比较低效
- ▶ WA, NT, Q, NSW, V, SA, T 的顺序效率不高  
因为Q, NSW, V 都有两种选择，未来有可能与SA的选择矛盾
- ▶ WA, NT, SA, Q, NSW, V 的选择效率更高  
因为WA和NT确定后，SA 仅有一种合法取值



## ▶ 最少剩余值：选择值域最小的变量

- ▶ 失败优先原则：可选值最少的变量，最容易失败
- ▶ 放到前面早一些失败，避免生成无用节点（剪枝）
- ▶ 放到后面晚一些失败，虽然已经生成很多无用节点，还是避免不了回溯到前面的变量（差别可达上千倍）

## ▶ 最多约束项：选择约束最多的变量

- ▶ 约束仅考虑尚未赋值变量
- ▶ 受约束多意味着以后失败的可能性大



# 取值选择的顺序

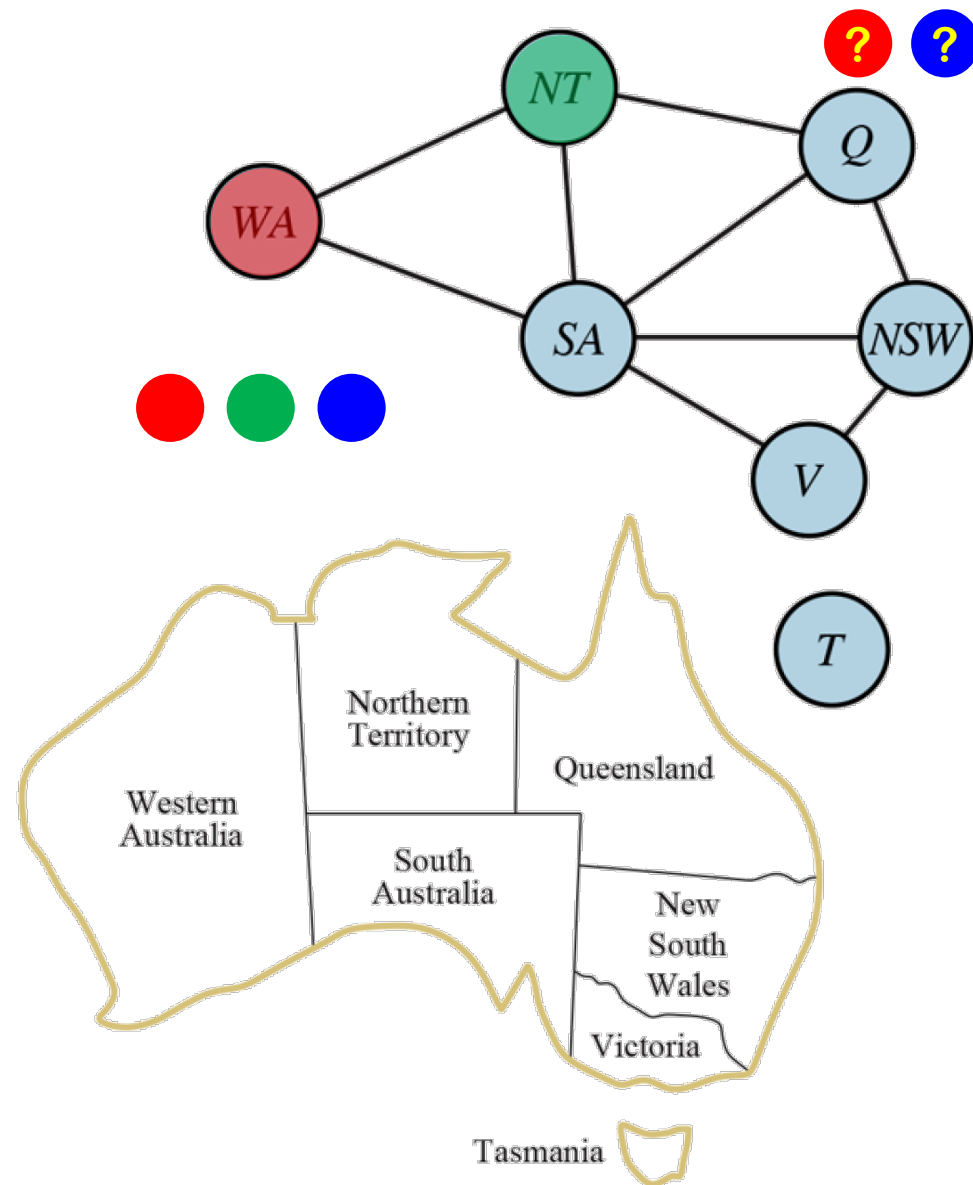
## ▶ 最少约束值：选择给邻居留下更多可能的值

- ▶ 这样邻居以后失败的可能性小
- ▶ 如果要求所有解，则该顺序不重要
- ▶ 如果问题没有解，则该顺序不重要

## ▶ 为什么对变量的选择是失败优先

### 而对值的选择是失败最后？

- ▶ 选择具有最少剩余值的变量，是希望通过早期的有效剪枝来减少搜索树中的结点数
- ▶ 而对于值的排序，目的是只需要找到一个解，因此首先选择最有可能的值是有意义的
- ▶ 如果要求获得所有的解而不是只找到一个解，值的排序就没有意义了



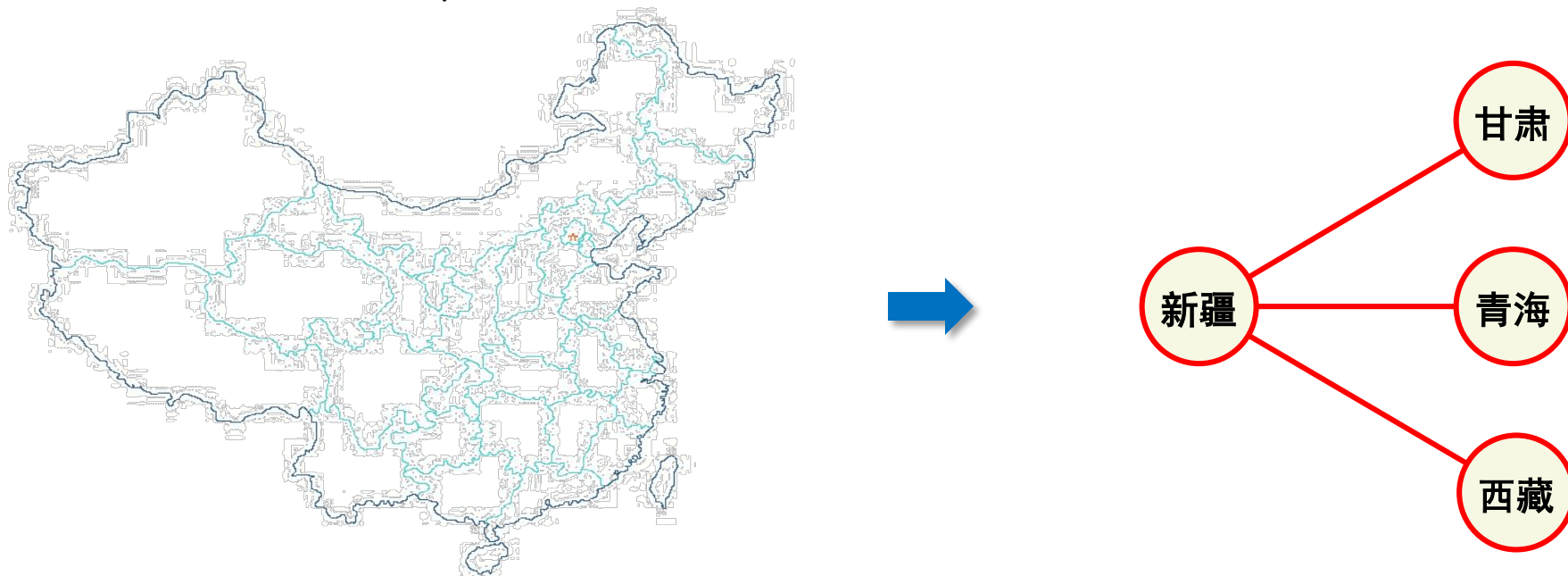
# 约束满足问题中的推理

Reasoning for CSP problems

|  |  |
|--|--|
|  |  |
|  |  |

# 节点相容

- ▶ 如果一个变量在其值域中的所有取值均满足其一元约束，则称该变量是节点相容的
- ▶ 如果所有变量均是节点相容的，则称问题对应的图是节点相容的
- ▶ 约束条件可以表示为一个图（网络），变量对应于节点，故称“节点相容”
- ▶ 不是指变量的某个取值，而是指变量值域中的所有可能取值
- ▶ 满足节点相容的变量，可以去掉其一元约束



# 边相容

- ▶ 如果一个变量在其值域中的所有取值均满足与该变量相关的二元约束，则称该变量是边相容的
- ▶ 如果所有变量均是边相容的，则称问题对应的图是边相容的
- ▶ 给定二元约束  $Y=X^2$ ,  $X, Y$  取正整数，则  $X$  是边相容的，而  $Y$  不是边相容的
- ▶ 给定二元约束  $Y=X^2$ ,  $Y$  取小于等于10的正整数，则  $X \in \{1, 2, 3\}$  时， $X$  是边相容的（此时  $Y$  是不是边相容的？）
- ▶ 给定二元约束  $Y=X^2$ ,  $Y \in \{1, 4, 9\}$ ,  $X \in \{1, 2, 3\}$ ，则  $X$  和  $Y$  都是边相容的
- ▶ 图着色问题，新疆  $\in \{R, G, B, Y\}$ ，青海  $\in \{R, G, B, Y\}$ ，这两个变量是不是边相容的？



# 边相容的检测算法

AC-3

**ALGORITHM** AC-3(csp)  
**INPUT** A CSP problem  
**OUTPUT** TRUE if all edges are consistent and FALSE otherwise

```
edge ← a set of edges initialized with all edges in csp
WHILE NOT edge.EMPTY() DO
     $(X_i, X_j) \leftarrow \text{edge.POP}()$ 
    IF REVISED(csp,  $X_i, X_j$ ) THEN
        IF size of  $\text{csp}.D_i = 0$  THEN RETURN false
        FOR EACH  $X_k$  in  $\text{NEIGHBORS}(X_i) - \{X_j\}$  DO
            add  $(X_k, X_i)$  to edge
RETURN true
```

```
revised ← false
FOR EACH  $x$  IN  $D_i$  DO
    IF no value  $y$  in  $D_j$  allows  $(x, y)$  to satisfy constraint between  $X_i$  and  $X_j$ 
        delete  $x$  from  $D_i$ 
        revised ← true
RETURN revised
```

**ALGORITHM** REVISED(csp,  $X_i, X_j$ )  
**OUTPUT** true iff the domain of  $X_i$  is revised

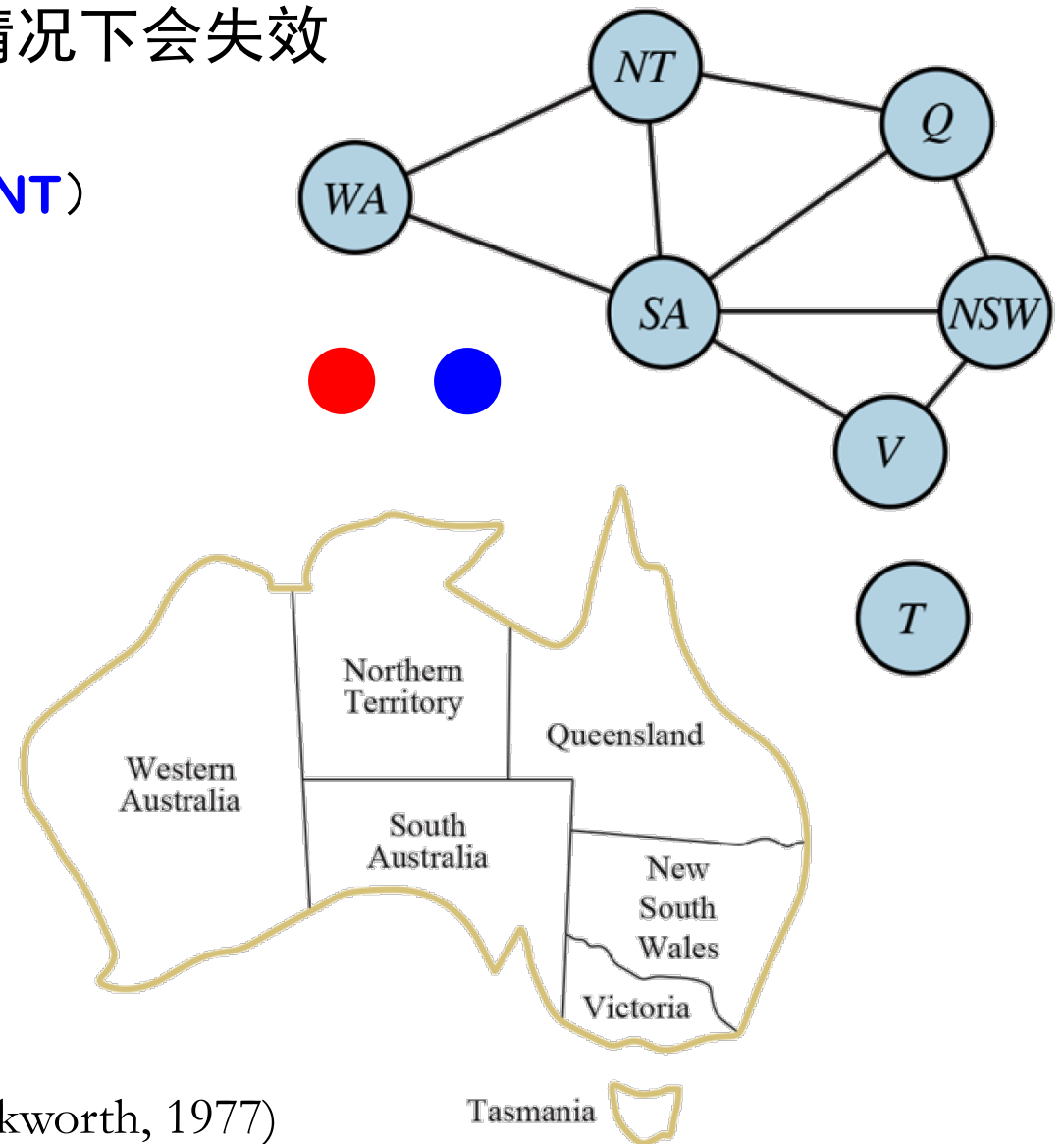
- ▶ 不断删除值域中不满足边相容的值，直到：
- ▶ 网络是边相容的
- ▶ 有变量值域为空待求解问题无解

Machworth (1977)

Consistency in networks of relations, Artificial Intelligence, 8(1): 99-118

# 路径相容

- ▶ 边相容仅涉及两个变量之间的关系，在有些情况下会失效
  - ▶ 澳大利亚地图用红蓝两种颜色着色，很明显无解
  - ▶ 但进行边相容检查起不到任何作用（例如 **WA, SA, NT**）
  - ▶ 需要引入更高阶次的检查
- ▶ 如果对两个变量  $\{X_i, X_j\}$  的每一个相容赋值，第三个变量  $X_m$  都有合适的取值，同时使得  $\{X_i, X_m\}$  和  $\{X_m, X_j\}$  是边相容的，则称  $\{X_i, X_j\}$  对于  $X_m$  是路径相容的
- ▶ 好像一条从  $X_i$  到  $X_m$  再到  $X_j$  的路径
- ▶ 考虑 **(WA, SA)** 对于 **NT** 的路径相容性
  - ▶ 很明显两种颜色是不可能做到相容的
  - ▶ 因此该问题无解
- ▶ 有一个路径相容检测算法 **PC-2** (类似 **AC-3**, Mackworth, 1977)





- ▶ 如果对任何  $k-1$  个变量的相容赋值, 第  $k$  个变量总能被赋予一个和它们相容的值, 则称这  $k-1$  个变量对于第  $k$  个变量是  $k$  相容的
  - ▶ 1—相容就是节点相容
  - ▶ 2—相容就是边相容
  - ▶ 3—相容就是路径相容
- ▶ 如果一个问题是  $k$ —相容的,  $(k-1)$ —相容的,  $(k-2)$ —相容的, ..., 1—相容的, 则称该问题是强  $k$ —相容的
- ▶ 强相容的问题能够较快地找到解
  - ▶ 1—相容: 对  $X_1$  能够找到一个相容的值
  - ▶ 2—相容: 对  $X_2$  能够找到一个与  $X_1$  相容的值
  - ▶ 3—相容: 对  $X_3$  能够找到一个与  $X_1, X_2$  相容的值
  - ▶ ...
  - ▶  $k$ —相容: 对  $X_k$  能够找到一个与  $X_1, \dots, X_{k-1}$  相容的值

# 全局约束

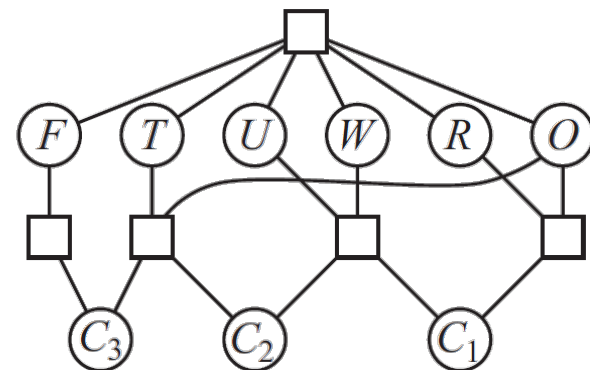
- 涉及多个变量（不一定是全部变量）的约束，可以用专门的高效算法检测相容性
- 所有变量必须取不同的值（**Alldiff**）

- 移除一个单值变量
- 从其他变量的值域中删除该值
- 重复该过程直到没有单值变量
- 如果出现有变量的值域为空集
- 如果变量个数大于值域的大小



不相容

$$\begin{array}{r} T \ W \ O \\ + \ T \ W \ O \\ \hline F \ O \ U \ R \end{array}$$



|   | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| A |   |   | 3 |   | 2 |   | 6 |   |   |
| B | 9 |   |   | 3 |   | 5 |   |   | 1 |
| C |   |   | 1 | 8 |   | 6 | 4 |   |   |
| D |   |   | 8 | 1 |   | 2 | 9 |   |   |
| E | 7 |   |   |   |   |   |   |   | 8 |
| F |   |   | 6 | 7 |   | 8 | 2 |   |   |
| G |   |   | 2 | 6 |   | 9 | 5 |   |   |
| H | 8 |   |   | 2 |   | 3 |   |   | 9 |
| I |   |   | 5 |   | 1 |   | 3 |   |   |



|   | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| A | 4 | 8 | 3 | 9 | 2 | 1 | 6 | 5 | 7 |
| B | 9 | 6 | 7 | 3 | 4 | 5 | 8 | 2 | 1 |
| C | 2 | 5 | 1 | 8 | 7 | 6 | 4 | 9 | 3 |
| D | 5 | 4 | 8 | 1 | 3 | 2 | 9 | 7 | 6 |
| E | 7 | 2 | 9 | 5 | 6 | 4 | 1 | 3 | 8 |
| F | 1 | 3 | 6 | 7 | 9 | 8 | 2 | 4 | 5 |
| G | 3 | 7 | 2 | 6 | 8 | 9 | 5 | 1 | 4 |
| H | 8 | 1 | 4 | 2 | 5 | 3 | 7 | 6 | 9 |
| I | 6 | 9 | 5 | 4 | 1 | 7 | 3 | 8 | 2 |

# 推理举例

- ▶ 81个变量  $A_1, \dots, A_9; \dots; I_1, \dots, I_9$
- ▶ 27个 Alldiff 约束：行、列、方格
- ▶ 每个 Alldiff 可以转化为36个二元约束  
可以用边相容来检测

- ▶  $E_6$ : {1, 2, 3, 4, 5, 6, 7, 8, 9}
- ▶ 第E行: {1, 2, 3, 4, 5, 6, 7, 8, 9}
- ▶ 第6列: {1, 2, 3, 4, 5, 6, 7, 8, 9}
- ▶ 所在方格: {1, 2, 3, **4**, 5, 6, 7, 8, 9}

- ▶  $I_6$ : {1, 2, 3, 4, 5, 6, 7, 8, 9}
- ▶ 第I行: {1, 2, 3, 4, 5, 6, 7, 8, 9}
- ▶ 第6列: {1, 2, 3, 4, 5, 6, **7**, 8, 9}



|   | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| A |   |   | 3 |   | 2 |   | 6 |   |   |
| B | 9 |   |   | 3 |   | 5 |   |   | 1 |
| C |   |   | 1 | 8 |   | 6 | 4 |   |   |
| D |   |   | 8 | 1 |   | 2 | 9 |   |   |
| E | 7 |   |   |   |   |   |   |   | 8 |
| F |   |   | 6 | 7 |   | 8 | 2 |   |   |
| G |   |   | 2 | 6 |   | 9 | 5 |   |   |
| H | 8 |   |   | 2 |   | 3 |   |   | 9 |
| I |   |   | 5 |   | 1 |   | 3 |   |   |

|   | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 |
|---|---|---|---|---|---|---|---|---|---|
| A | 4 | 8 | 3 | 9 | 2 | 1 | 6 | 5 | 7 |
| B | 9 | 6 | 7 | 3 | 4 | 5 | 8 | 2 | 1 |
| C | 2 | 5 | 1 | 8 | 7 | 6 | 4 | 9 | 3 |
| D | 5 | 4 | 8 | 1 | 3 | 2 | 9 | 7 | 6 |
| E | 7 | 2 | 9 | 5 | 6 | 4 | 1 | 3 | 8 |
| F | 1 | 3 | 6 | 7 | 9 | 8 | 2 | 4 | 5 |
| G | 3 | 7 | 2 | 6 | 8 | 9 | 5 | 1 | 4 |
| H | 8 | 1 | 4 | 2 | 5 | 3 | 7 | 6 | 9 |
| I | 6 | 9 | 5 | 4 | 1 | 7 | 3 | 8 | 2 |

# 推理与搜索交替进行

Searching for CSP problems

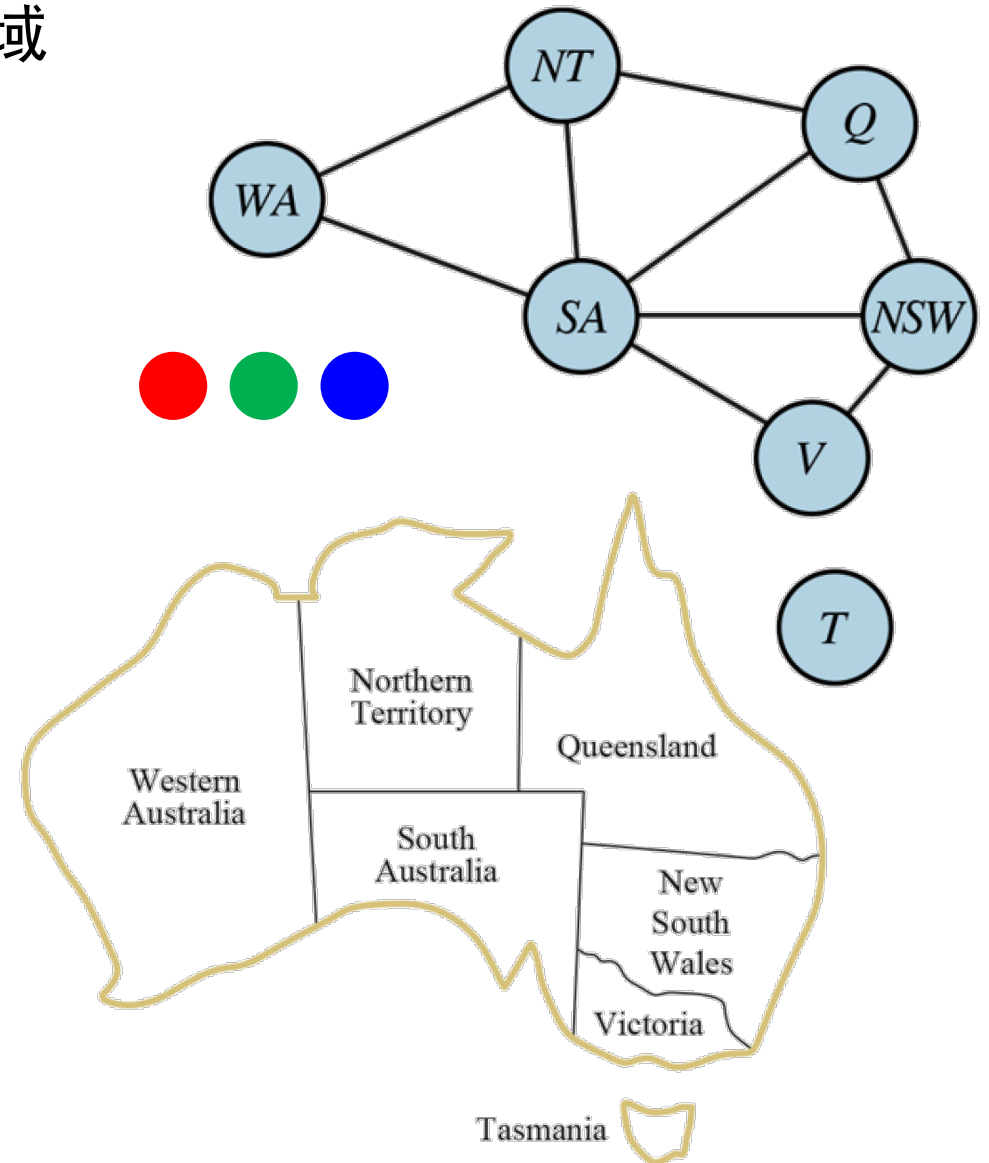
|  |  |
|--|--|
|  |  |
|  |  |

# 搜索后推理

- 每次决定某变量的取值后，推理其邻居变量的值域
- 前向检查：在变量赋值后对它进行边相容检查**
- 删除与该变量相关的未赋值变量不相容的值

|                      | <i>WA</i>                                    | <i>NT</i>                                    | <i>Q</i>                                     | <i>NSW</i>                                   | <i>V</i>                                     | <i>SA</i>                                    | <i>T</i>                                     |                                              |
|----------------------|----------------------------------------------|----------------------------------------------|----------------------------------------------|----------------------------------------------|----------------------------------------------|----------------------------------------------|----------------------------------------------|----------------------------------------------|
| Initial domains      | <div><div></div><div></div><div></div></div> | <div><div></div><div></div><div></div></div> | <div><div></div><div></div><div></div></div> | <div><div></div><div></div><div></div></div> | <div><div></div><div></div><div></div></div> | <div><div></div><div></div><div></div></div> | <div><div></div><div></div><div></div></div> | <div><div></div><div></div><div></div></div> |
| After <i>WA=red</i>  | <div><div></div><div></div><div></div></div> | <div><div></div><div></div><div></div></div> | <div><div></div><div></div><div></div></div> | <div><div></div><div></div><div></div></div> | <div><div></div><div></div><div></div></div> | <div><div></div><div></div><div></div></div> | <div><div></div><div></div><div></div></div> | <div><div></div><div></div><div></div></div> |
| After <i>Q=green</i> | <div><div></div><div></div><div></div></div> | <div><div></div><div></div><div></div></div> | <div><div></div><div></div><div></div></div> | <div><div></div><div></div><div></div></div> | <div><div></div><div></div><div></div></div> | <div><div></div><div></div><div></div></div> | <div><div></div><div></div><div></div></div> | <div><div></div><div></div><div></div></div> |
| After <i>V=blue</i>  | <div><div></div><div></div><div></div></div> | <div><div></div><div></div><div></div></div> | <div><div></div><div></div><div></div></div> | <div><div></div><div></div><div></div></div> | <div><div></div><div></div><div></div></div> | <div><div></div><div></div><div></div></div> | <div><div></div><div></div><div></div></div> | <div><div></div><div></div><div></div></div> |

- 最小剩余值选择与前向检查联合使用
  - 在选择 WA 后，应该选择 NT 和 SA，而不是 Q
  - 前向检查提供了进行最小剩余值选择的必要信息



## ▶ 时序回溯：需要回溯时，退回到前一个变量，尝试其下一个取值

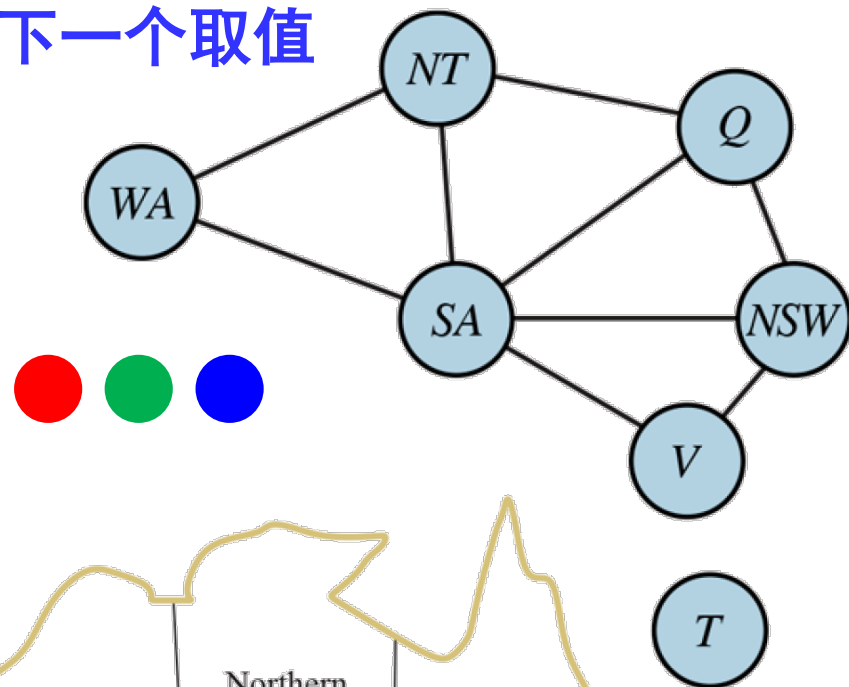
- ▶ 按照 Q, NSW, V, T, SA, WA, NT 的顺序
- ▶ {Q=red, NSW=green, V=blue, T=red} 时，考虑 SA
- ▶ 需要回溯，退回到 T，但其实并无帮助

## ▶ 智能回溯：需要回溯时，退回到产生冲突的变量

- ▶ 上面例子，可以跳过 T，直接退回到 V

## ▶ 冲突集：可能与某变量产生冲突的变量

- ▶ 冲突集可以直接由前向检验获得
- ▶ 当基于赋值  $X = x$  的前向检验删除变量  $Y$  值域中的值时应该把  $X = x$  加入到  $Y$  的冲突集
- ▶ 如果此时从  $Y$  的值域中删除的是最后一个值，应该把  $Y$  的冲突集中的每个赋值加入到  $X$  的冲突集



# 回溯搜索算法

# Backtracking search

**ALGORITHM** BACKTRACK(assignment, csp)  
**INPUT** An assignment of the parameters and a CSP problem  
**OUTPUT** A solution or failure

```
IF assignment is complete THEN RETURN assignment
var ← SELECTUNASSIGNEDVARIABLE(csp)
FOR EACH value IN ORDERDOMAINVALUES(var, assignment, csp) DO
  IF value is consistent with assignment THEN
    add {var = value} to assignment
    inferences ← INFERENCE(csp, var, value)
    IF inferences ≠ failure THEN
      add inferences to assignment
      result ← BACKTRACK(assignment, csp)
      IF result ≠ failure THEN RETURN result
  Remove {var = value} and inferences from assignment
RETURN failure
```

静态选择  
最少剩余值  
最多约束项

最少约束值

时序回溯  
智能回溯

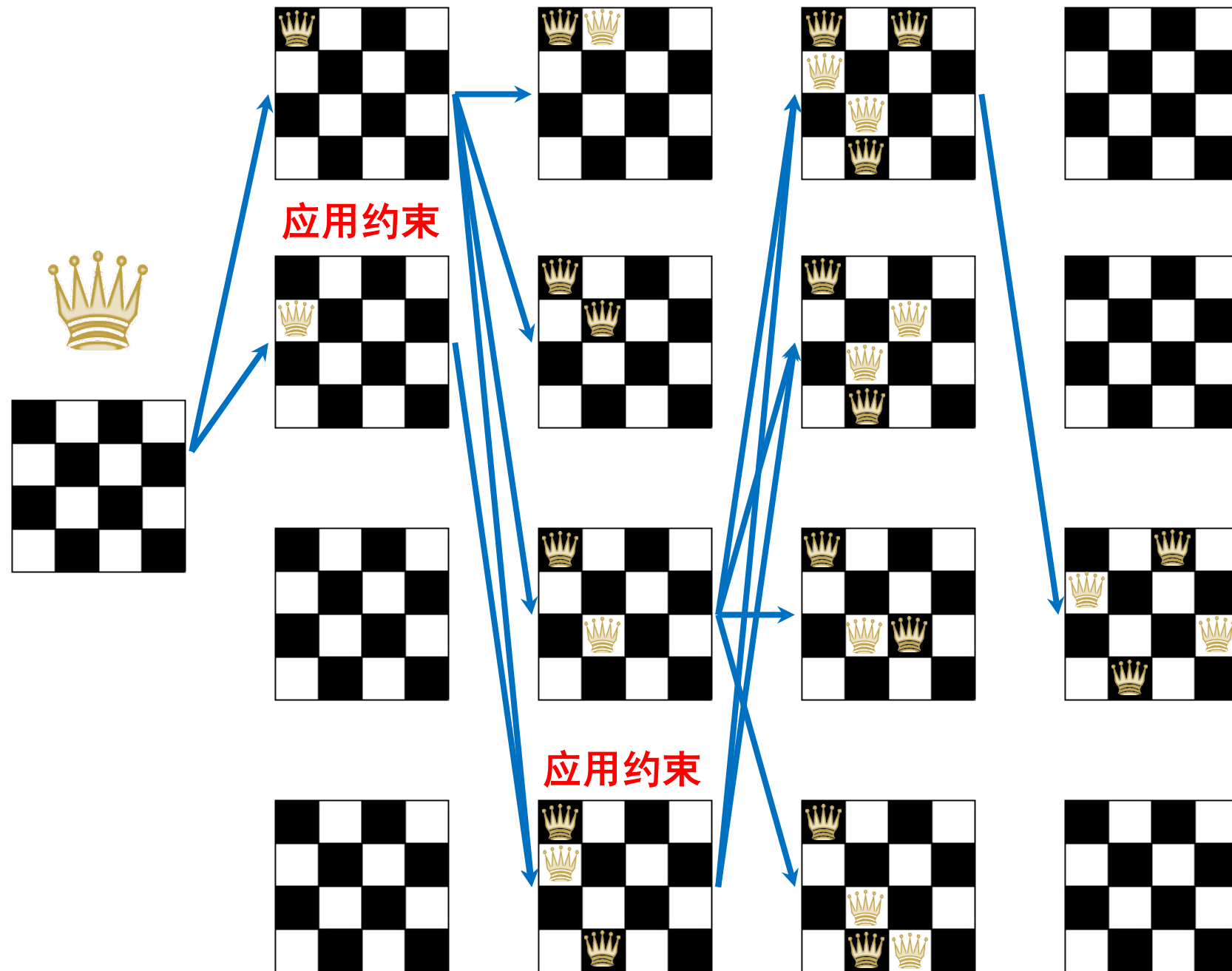
**ALGORITHM** BACKTRACKING-SEARCH(csp)  
**OUTPUT** A solution or failure

**RETURN** BACKTRACK({}, csp)



# 回溯搜索举例

## ► 8 皇后问题



# 局部搜索

Local Search

|  |  |
|--|--|
|  |  |
|  |  |

# 局部搜索

- 目的：搜索解的状态
- 原理：从随机初始状态逐步改进直到发现解

## 8 皇后问题

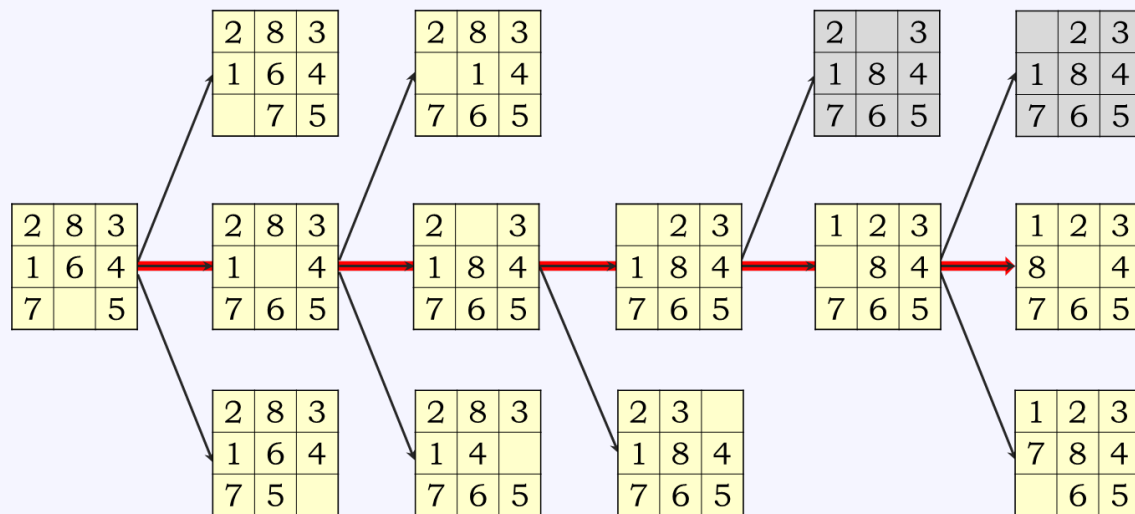
- 初始状态：为每个皇后随机指定一行，可能存在冲突
- 搜索过程：把一个皇后移动到另一行，尝试消除冲突
- 重复该消除过程，直到冲突全都消除，获得问题的解

### 随机产生的八皇后问题

14%的情况能找到解，平均4步能找到解  
86%的情况会被卡住，平均3步会被卡住

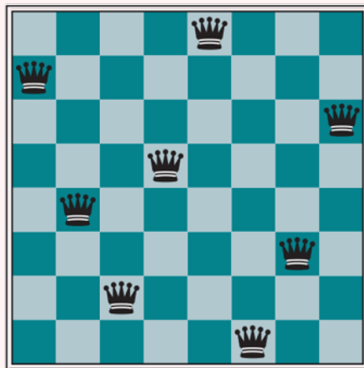
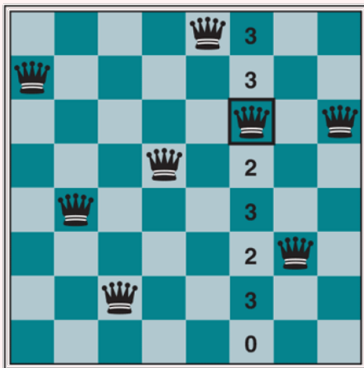
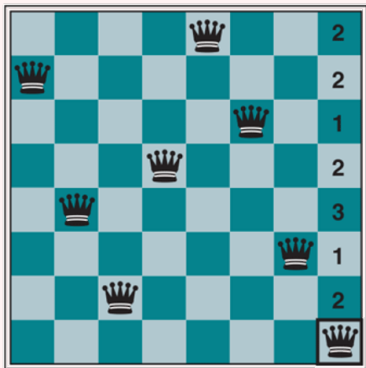
# 全局搜索

- 目的：搜索解的路径
- 原理：从初始状态进行扩展直到目标状态
- 方法：深度优先搜索  
宽度优先搜索  
一致代价搜索  
贪婪最佳优先  
**A\* 算法**
- 特点：完备、复杂、指数复杂度



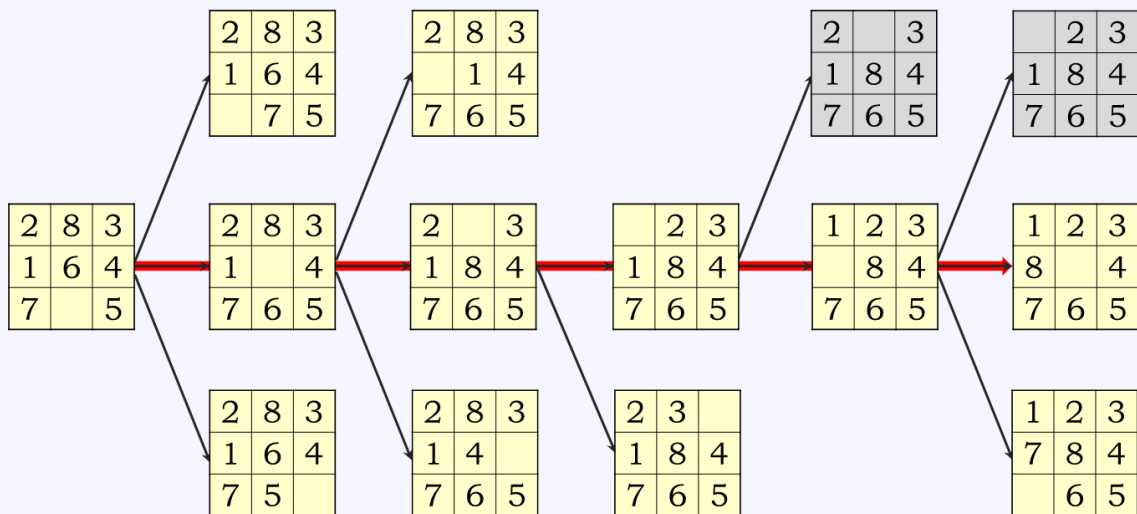
# 局部搜索

- ▶ 目的：搜索解的状态
- ▶ 原理：从随机初始状态逐步改进直到发现解
- ▶ 特点：时间效率极高  
空间效率极高  
能够求解大规模问题
- ▶ 方法：最小冲突数  
爬山法  
模拟退火  
遗传算法  
梯度下降



# 全局搜索

- ▶ 目的：搜索解的路径
- ▶ 原理：从初始状态进行扩展直到目标状态
- ▶ 方法：深度优先搜索  
宽度优先搜索  
一致代价搜索  
贪婪最佳优先  
**A\* 算法**
- ▶ 特点：完备、复杂、指数复杂度



# 最小冲突数法

# Minimum number of conflicts

**ALGORITHM** MINCONFLICTS(*csp*, *max-steps*)

**INPUT** An assignment of the parameters and a CSP problem

**OUTPUT** A solution, or failure

current  $\leftarrow$  an initial complete assignment for *csp*

**FOR** *i* = 1 **TO** *max\_steps* **DO**

**IF** current is a solution for *csp* **THEN RETURN** current

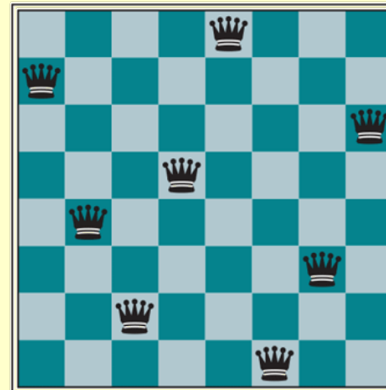
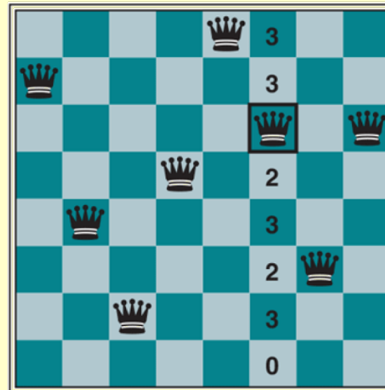
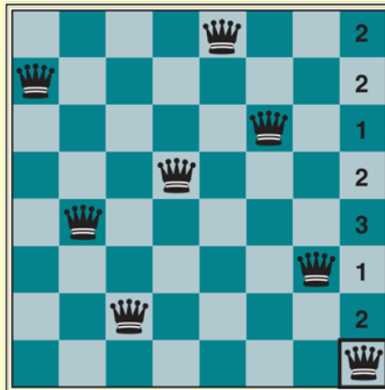
    var  $\leftarrow$  a random chosen conflicted variable for *csp*

    value  $\leftarrow$  the value *v* for var that minimizes CONFLICTS(var, *v*, current, *csp*)

    set var = value in current

**RETURN** failure

随机选择一个变量  
取最小化冲突的值

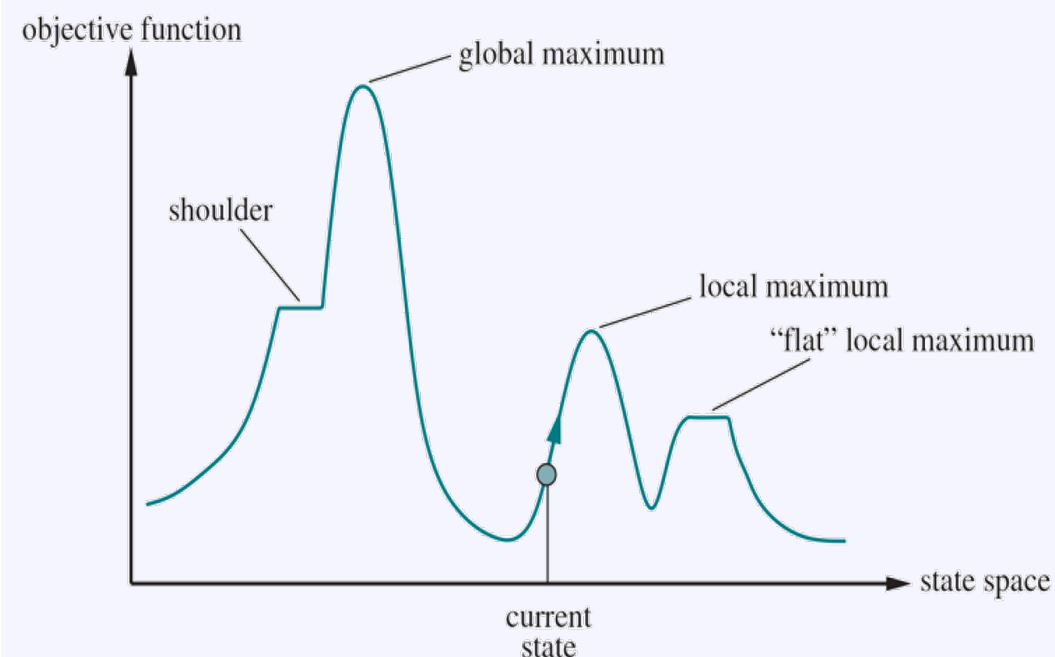
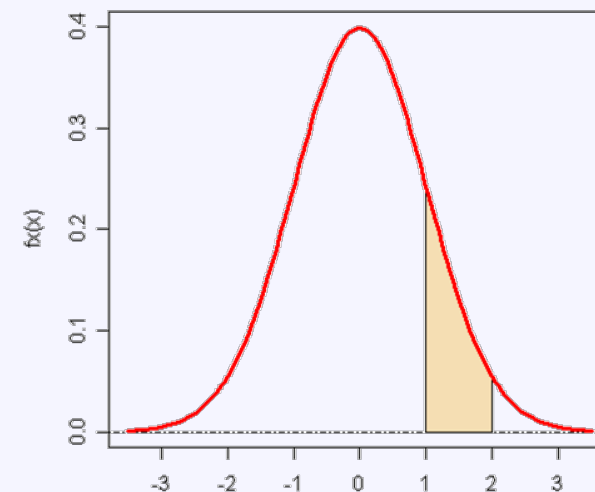


# 爬山法

- ▶ 优化问题：搜索能使目标函数最大的状态
- ▶ 贪婪局部搜索：
  - ▶ 从初始状态开始，不断向函数值增加的状态移动

|                  |                                    |
|------------------|------------------------------------|
| <b>ALGORITHM</b> | <b>HILL-CLIMBING(problem)</b>      |
| <b>INPUT</b>     | A problem to be optimized          |
| <b>OUTPUT</b>    | A solution that is a local maximum |

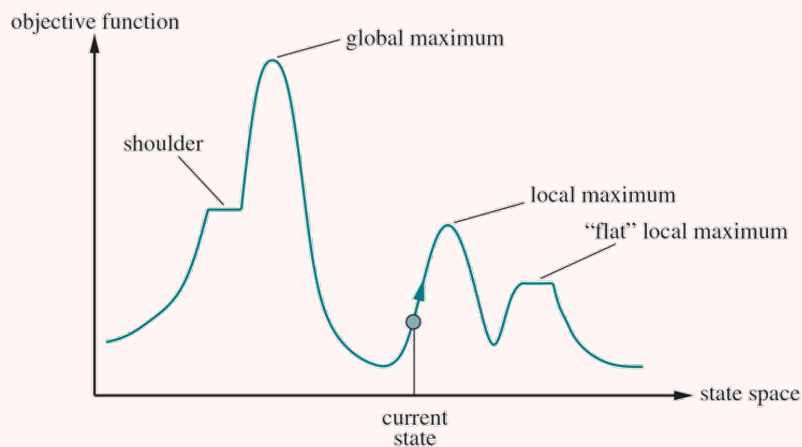
```
current ← MAKE-NODE(problem.INITIAL-STATE)
WHILE true DO
    neighbor ← a highest-valued successor of current
    IF neighbor.VALUE ≤ current.VALUE THEN
        RETURN current.STATE
    current ← neighbor
```



# 可能遇到的困难

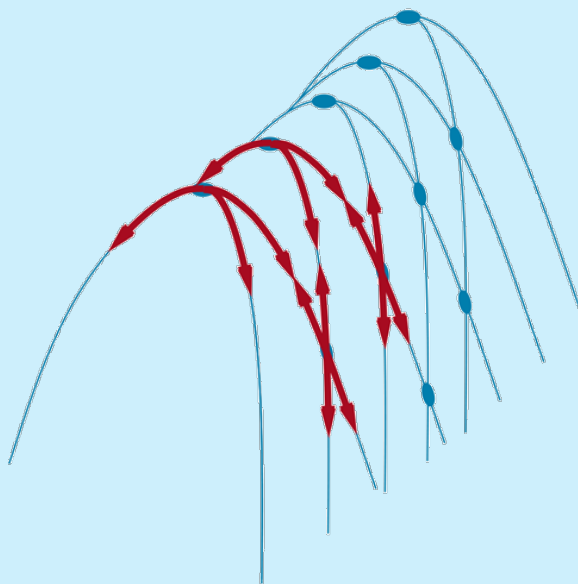
## 局部极值

- ▶ 比附近状态函数值都大
- ▶ 算法会陷入局部极大值



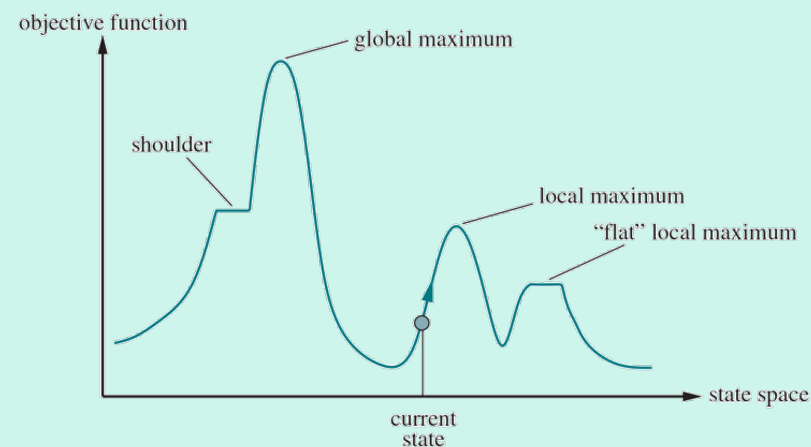
## 山脊

- ▶ 一系列的局部极大值
- ▶ 算法难以处理这类情况



## 高原

- ▶ 一块平的局部极大值
- ▶ 算法会迷失方向





## 随机移动

- ▶ 根据上山移动的陡峭程度随机选择下一步
- ▶ 收敛速度慢，但有可能避开局部极大值点

## 随机重启

- ▶ 被卡住时随机生成初始状态从头开始
- ▶ **完备的概率接近于 1**
- ▶ 300万个皇后找到解的时间不超过1分钟

## 局部搜索

## 模拟退火

- ▶ 不仅允许上山，还以一定概率允许下山
- ▶ 上世纪80年代广泛应用于求解VLSI布局
- ▶ 广泛应用于工厂调度和求解大型优化任务
- ▶ 理论基础：马尔可夫链蒙特卡洛

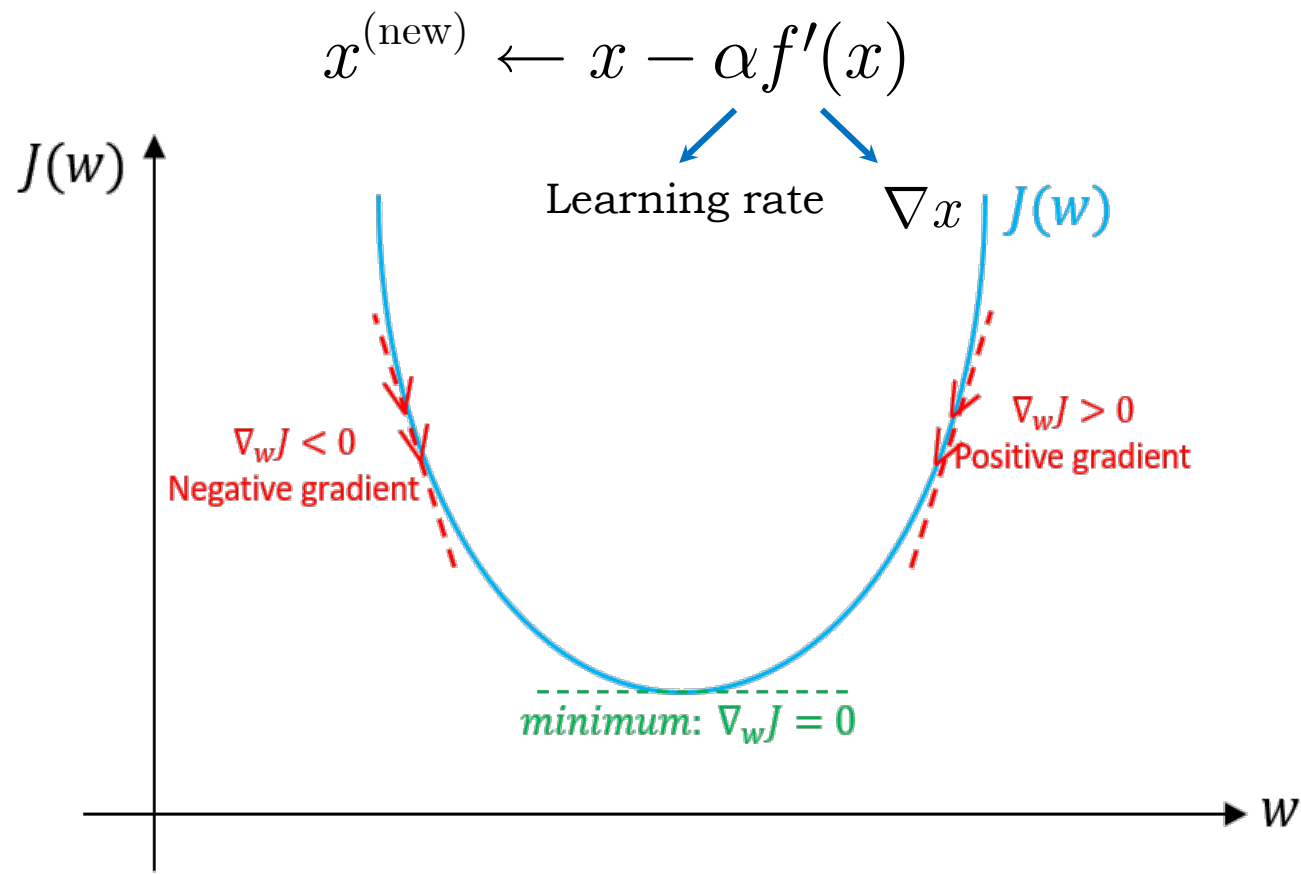
Markov Chain Monte Carlo (MCMC)

## 局部束搜索

- ▶ 从多个状态开始并行地进行局部搜索
- ▶ 每次从新生成的状态中挑选等量最好的替换当前状态
- ▶ 变体：随机束搜索，遗传算法

# 梯度下降

- ▶ 凸函数极小点导数为零
- ▶ 大于极小点时导数为正 → 往负导数方向修正
- ▶ 小于极小点时导数为负 → 往负导数方向修正



- ▶ 泰勒级数展开

$$f(x) = f(a) + \frac{f'(a)}{1!}(x-a) + \frac{f''(a)}{2!}(x-a)^2 + \dots$$

- ▶ 一阶近似

$$f(x + \Delta x) \approx f(x) + f'(x)\Delta x$$

- ▶ 梯度下降

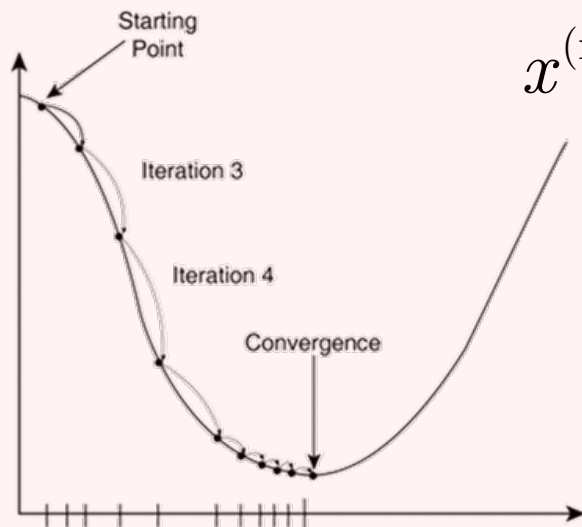
$$\underline{f(x^{(\text{new})}) = f(x - \alpha f'(x)) \approx f(x) - \alpha (f'(x))^2 \leq f(x)}$$



Cauchy, 1847

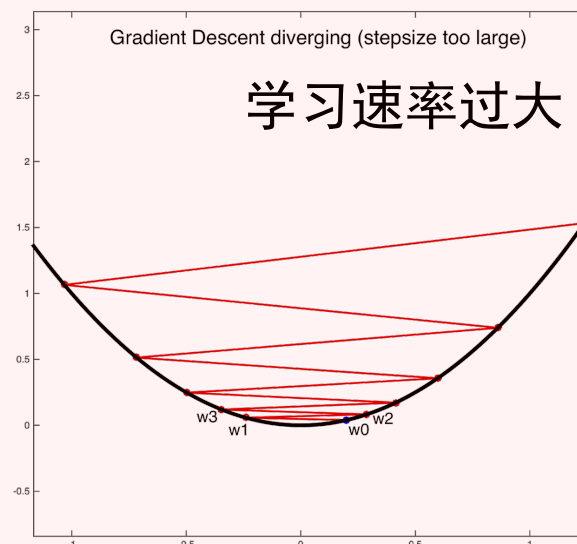
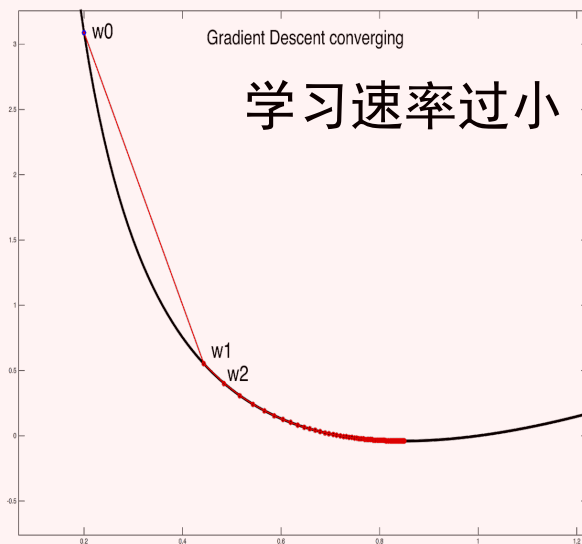
# 学习速率

## 一维



$$x^{(\text{new})} \leftarrow x - \alpha f'(x)$$

Learning rate  $\nabla x$



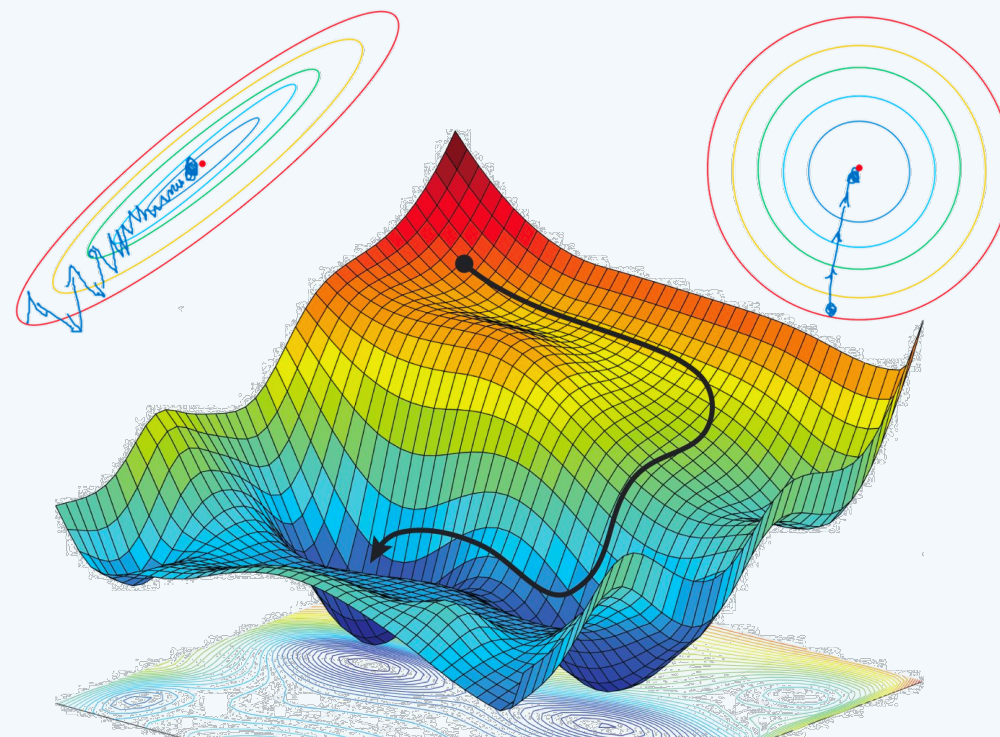
## 二维

$$\mathbf{x}^{(\text{new})} \leftarrow \mathbf{x} - \alpha \nabla \mathbf{x}$$
$$\nabla \mathbf{x} = \begin{pmatrix} \frac{\partial f(\mathbf{x})}{\partial x_1} \\ \frac{\partial f(\mathbf{x})}{\partial x_2} \end{pmatrix}$$



Unnormalized

Normalized



# 随机梯度下降

# Stochastic Gradient Descent

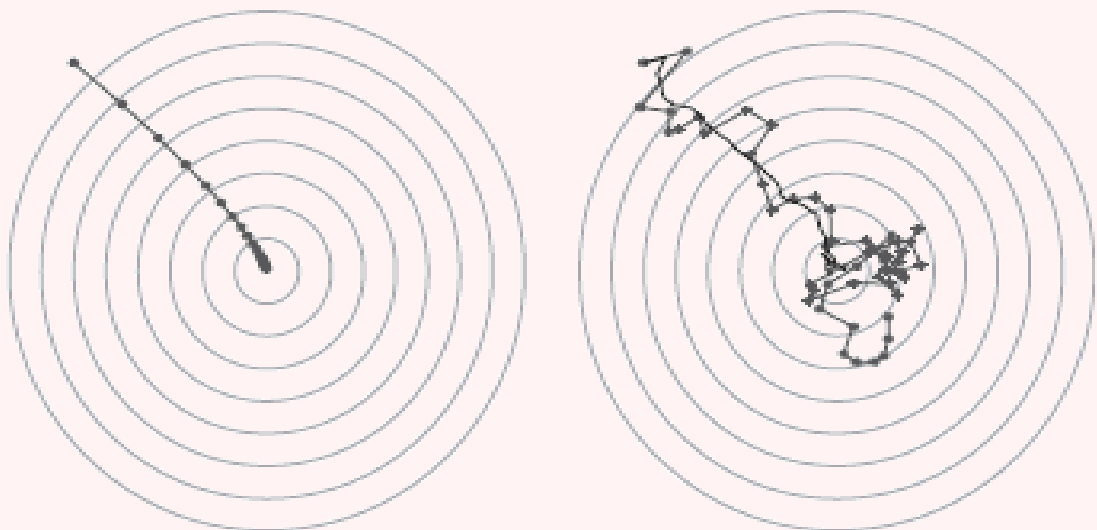
- ▶ 计算梯度的负担太重

$$\min -\frac{1}{n} \sum_{i=1}^n (y_i \log \pi(x_i) + (1 - y_i) \log(1 - \pi(x_i)))$$

需要把整个数据集算一遍

- ▶ 如何高效地估计梯度

- ▶ 随机取数据的一个子集并计算梯度
- ▶ 极端情况一次用一个样例计算梯度



- ▶ 一次用一个样例来计算梯度

$$\min -(y_i \log \pi(x_i) + (1 - y_i) \log(1 - \pi(x_i)))$$

只用算一个样例

- ▶ 主要优点

- ▶ 计算简单
- ▶ 计算量小

- ▶ 保证随机

- ▶ 离线计算时样例的选取不要依照一定顺序
- ▶ 可以用于在线计算

## 了解

- ▶ 约束满足
  - ▶ 求解过程中的推理
- ▶ 局部搜索
  - ▶ 模拟退火
  - ▶ 遗传算法

## 理解

- ▶ 约束满足
- ▶ 局部搜索
  - ▶ 随机移动
  - ▶ 随机重启

## 掌握

- ▶ 约束满足
  - ▶ 回溯算法
- ▶ 局部搜索
  - ▶ 爬山法
  - ▶ 梯度下降
  - ▶ 随机梯度下降

**Thank you very much!**